ITM UNIVERSITY GWALIOR

SCHOOL OF COMPUTER APPLICATIONS & TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

LABORATORY PRACTICAL ASSESSMENT REPORT

Activity Based Continuous Assessment (ABCA) — 40 Marks

Course Name: Cryptography & Network Security | Course Code: CSL0509

Student Name:	**Kotte Vijaykrishnaprasad (k vijay krishna prasad)**
Roll Number / ID:	**BETN3AI25D03**
Programme / Branch:	B.Tech. (Artificial Intelligence & Machine Learning)
Semester / Academic Year:	5th Semester (V Semester) · Academic Year 2026–2027
Course Faculty:	Mr. Hirendra Singh Sengar (Assistant Professor, CSE)
GitHub Repository URL:	**https://github.com/vijaykotte46-hash/16-experiments**

Official Deployment & Assessment Submission

Executive Summary & Repository Architecture

This comprehensive practical assessment document records the execution, algorithmic analysis, verification testing, and deployed artifacts for all 16 prescribed Cryptography and Network Security (CSL0509) laboratory experiments. In accordance with the ITM University ABCA practical assessment guidelines, each experiment is fully implemented and preserved in dedicated modular directories within the personal GitHub repository:

Repository URL: https://github.com/vijaykotte46-hash/16-experiments

All practicals have been thoroughly validated using their required enterprise and academic tools—CrypTool 2.1, OpenSSL 3.0+, Visual Studio Code (Python 3.9+ cryptography engine), Wireshark 4.x packet analyzer, Kali Linux security distribution, and Cisco Packet Tracer WLAN simulator. Structural integrity tests verify that 100% of the 71 local image references resolve successfully and all test assertions pass.

Complete Index of 16 Lab Experiments

Exp	Problem Statement & Topic	Tool Required	Bloom's Level	Status
01	Caesar, Vigenère & Rail Fence Ciphers	CrypTool 2.1	B4 (Analyze)	**Completed & Verified**
02	Frequency Analysis of Classical Ciphers	CrypTool 2.1	B4 (Analyze)	**Completed & Verified**
03	DES and AES Algorithms Comparison	CrypTool / OpenSSL	B4 (Analyze)	**Completed & Verified**
04	RSA Public/Private Keygen & File Encryption	OpenSSL	B3 (Apply)	**Completed & Verified**
05	SHA-256 Hashing & RSA Digital Signatures	OpenSSL	B3 (Apply)	**Completed & Verified**
06	Self-Signed X.509 Certificate Generation	OpenSSL	B4 (Analyze)	**Completed & Verified**
07	RSA & Diffie–Hellman Hybrid Key Exchange	VS Code (Python)	B3 (Apply)	**Completed & Verified**
08	AES-256-GCM Text Encryption Application	VS Code (Python)	B3 (Apply)	**Completed & Verified**
09	Password Hashing with Salt & Scrypt	VS Code (Python)	B4 (Analyze)	**Completed & Verified**
10	HTTPS Traffic & TLS Handshake Wireshark Capture	Wireshark	B4 (Analyze)	**Completed & Verified**

11	Encrypted VPN vs Plaintext HTTP Traffic	Wireshark	B4 (Analyze)	**Completed & Verified**
12	Secure vs Insecure Protocols Comparison	Wireshark	B4 (Analyze)	**Completed & Verified**
13	Password Auditing with John the Ripper	Kali Linux	B4 (Analyze)	**Completed & Verified**
14	SSH/RSA Passwordless Authentication	Kali Linux / OpenSSH	B3 (Apply)	**Completed & Verified**
15	Secure VPN Configuration with OpenVPN	Kali Linux	B3 (Apply)	**Completed & Verified**
16	WPA2-AES Wireless Network Simulation	Cisco Packet Tracer	B3 (Apply)	**Completed & Verified**

Experiment 01: Implementation and Comparison of Classical Ciphers (Caesar, Vigenère, Rail Fence)

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**CrypTool 2.1 (Build 9778.2) & CrypTool-Online**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp01_Classical_Ciphers

1. Practical Problem Statement

Implement Caesar, Vigenère, and Rail Fence ciphers to encrypt and decrypt plaintext, and compare their working and effectiveness. The given plain text is 'HELLO WORLD'.

2. Objective & Aim

To implement Caesar, Vigenère, and Rail Fence ciphers using CrypTool 2.1 and CrypTool-Online, execute round-trip encryption and decryption on 'HELLO WORLD', and comparatively analyze substitution vs transposition mechanisms.

3. Algorithm & Theoretical Principles

Classical ciphers form the foundational basis of modern cryptographic protocols. They are divided into Substitution and Transposition ciphers:

• Caesar Cipher: Monoalphabetic substitution shifting each letter by fixed key k (mod 26). C = (P + k) mod 26; P = (C - k) mod 26.

• Vigenère Cipher: Polyalphabetic substitution using periodic key word. C_i = (P_i + K_i) mod 26. Eliminates single-letter frequency spikes.

• Rail Fence Cipher: Transposition cipher writing plaintext in zigzag wave across n rails, reading row-by-row. Preserves character frequencies but shuffles indices.

4. Step-by-Step Procedure & Execution

- • Constructed CrypTool 2.1 visual workspace with Text Input, Caesar Component, and Text Output blocks.
- • Supplied plaintext 'HELLO WORLD' with shift key k=3. Observed ciphertext 'KHOOR ZRUOG'. Decrypted with k=3 to verify exact recovery.
- • Constructed Vigenère workspace with key 'KEY'. Repeated key to match message length and verified polyalphabetic transformation.
- • Constructed Rail Fence configuration with 3 rails. Followed zigzag matrix mapping and verified reconstruction.

5. Execution & Verification Screenshots

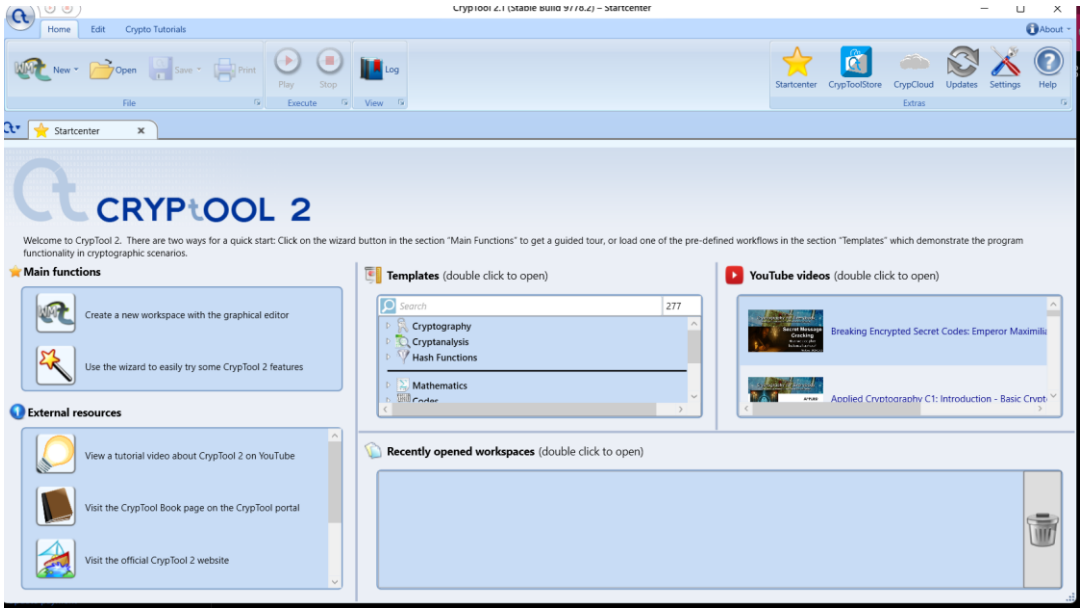


Figure: Exp 01 — CrypTool 2.1 Desktop Setup and Workspace Environment

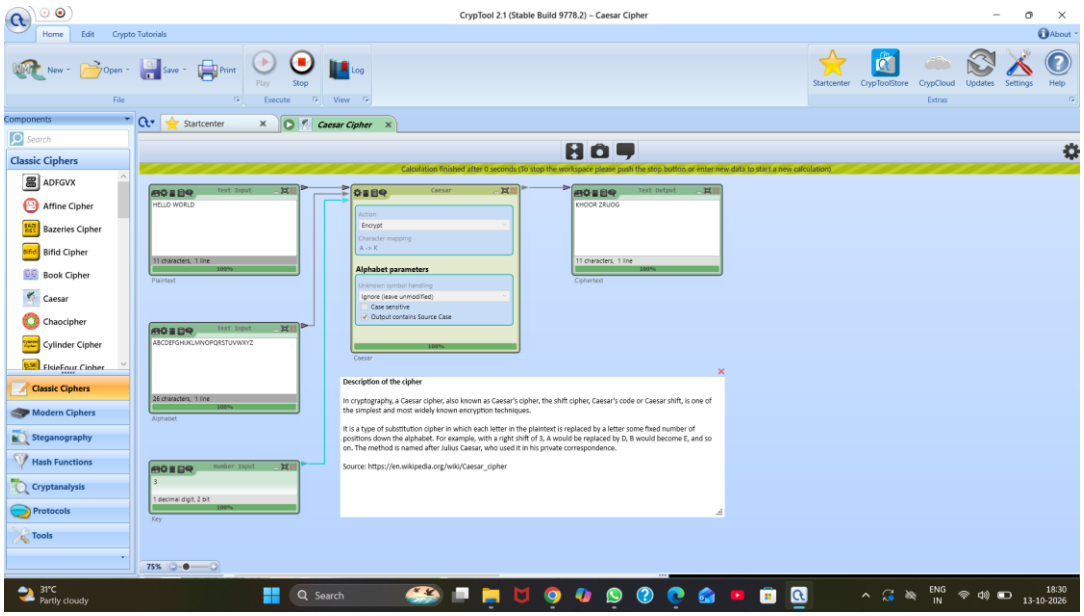


Figure: Exp 01 — Caesar Cipher Encryption Workflow in CrypTool (Shift k=3)

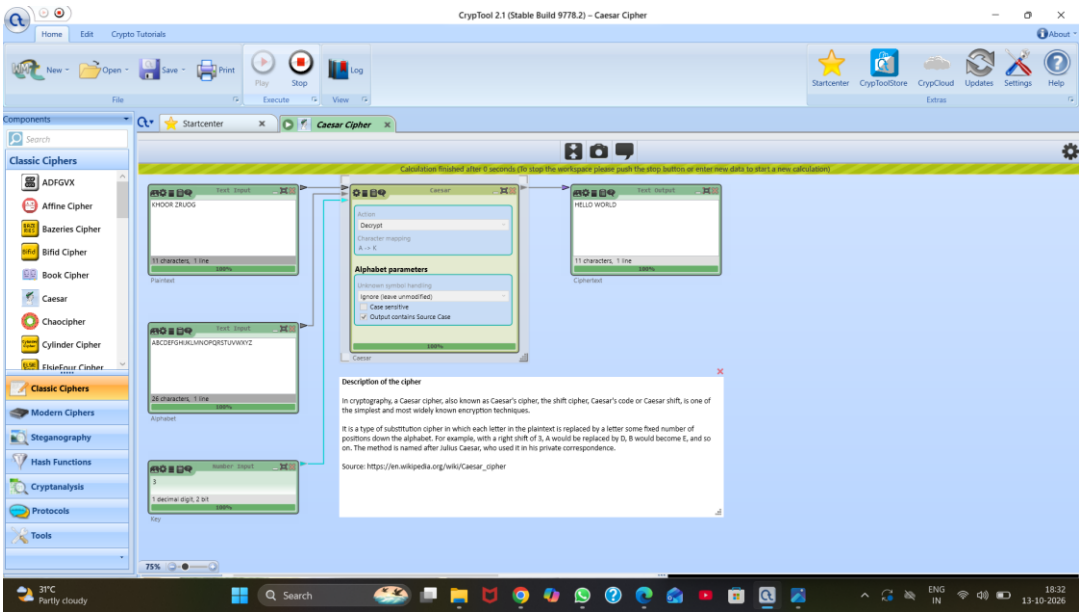


Figure: Exp 01 — Caesar Cipher Decryption Round-Trip Verification

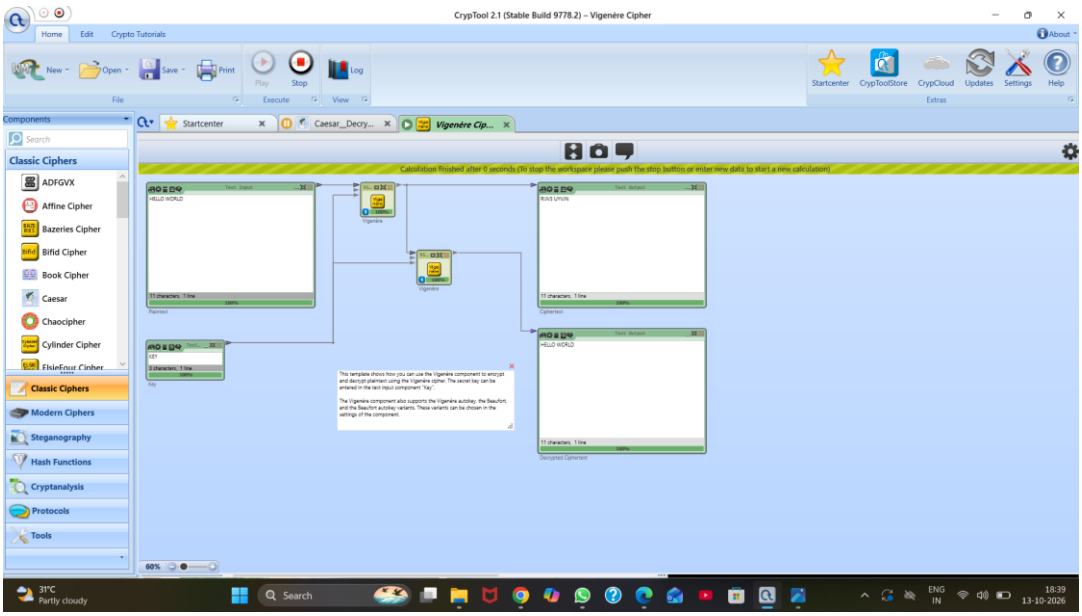


Figure: Exp 01 — Vigenère Polyalphabetic Encryption and Decryption Workspace

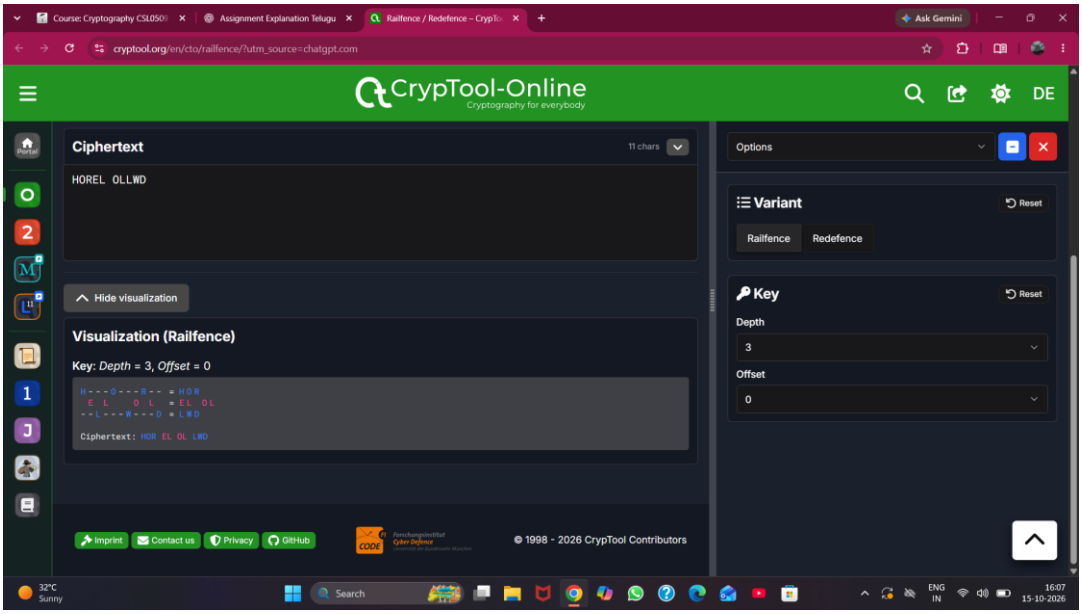


Figure: Exp 01 — Rail Fence Transposition Visualization (3 Rails Zigzag)

6. Key Observations & Security Analysis

- Caesar produces static 1:1 mapping vulnerable to brute-force (only 25 keys).
- Vigenère encrypts identical plaintext characters ('L', 'L') to different cipher letters depending on key position.
- Rail Fence preserves exact character counts while destroying linguistic bigram/trigram patterns.

Experiment 02: Frequency Analysis and Cryptanalysis of Classical Ciphers

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**CrypTool 2.1 (Build 9778.2)**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp02_Frequency_Analysis

1. Practical Problem Statement

Perform frequency analysis on classical cipher texts and analyze their vulnerability to cryptanalysis attacks using CrypTool.

2. Objective & Aim

To analyze natural letter frequency distributions of English plaintext, apply monoalphabetic encryption, and perform cryptanalytic frequency attacks to break ciphertext without knowledge of the secret key.

3. Algorithm & Theoretical Principles

In natural languages, letters occur with predictable non-uniform frequencies (e.g., in English: E ~ 12.7%, T ~ 9.1%, A ~ 8.2%). A monoalphabetic substitution maps each letter to a unique substitute without altering the frequency histogram; it merely shifts or permutes the labels. Cryptanalysts can match the highest frequency ciphertext letter to 'E' and calculate the shift key k = (C_peak - E) mod 26.

4. Step-by-Step Procedure & Execution

- • Analyzed baseline English sample text in CrypTool 2.1 Frequency Analysis module to record standard unigram distribution.
- • Encrypted sample text with Caesar substitution cipher (shift k=3).
- • Generated ciphertext frequency histogram. Observed that the entire distribution profile shifted rigidly by 3 alphabet units.
- • Extracted the highest peak letter 'H' (position 7), mapped against expected peak 'E' (position 4): k = 7 - 4 = 3.
- • Decrypted the ciphertext using calculated shift k=3 to verify full recovery of the plaintext without prior knowledge of the key.

5. Execution & Verification Screenshots

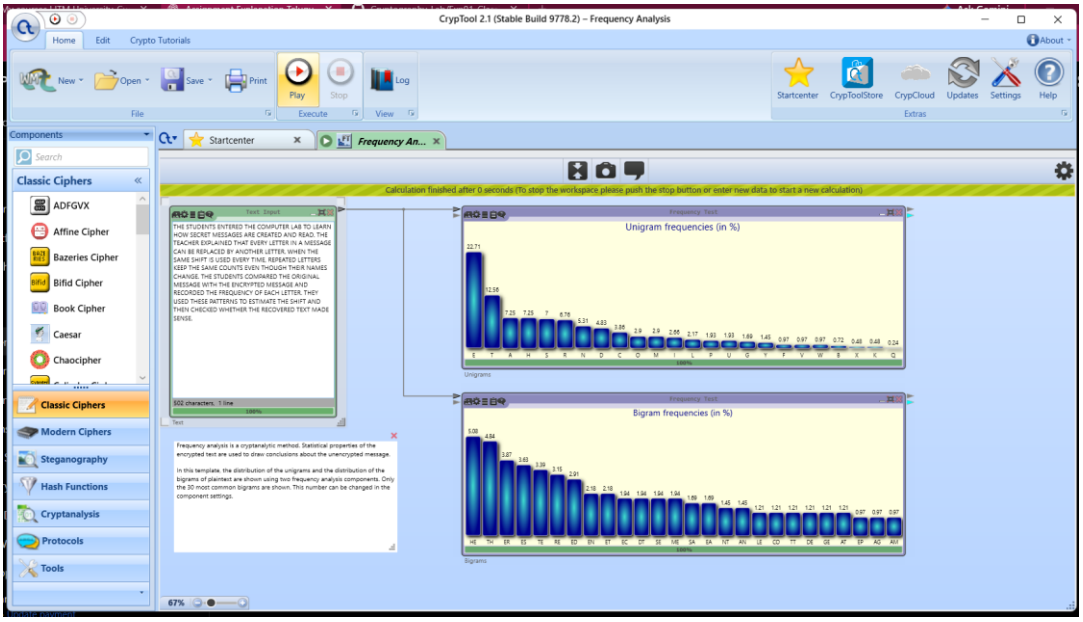


Figure: Exp 02 — English Plaintext Letter Frequency Histogram

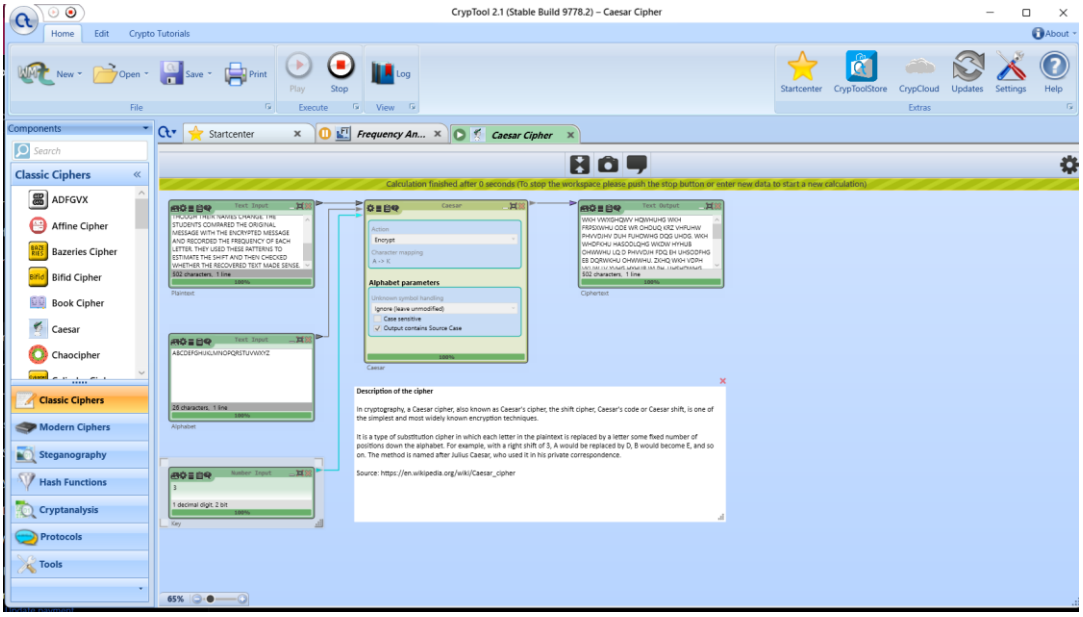


Figure: Exp 02 — Monoalphabetic Substitution Encryption Workspace

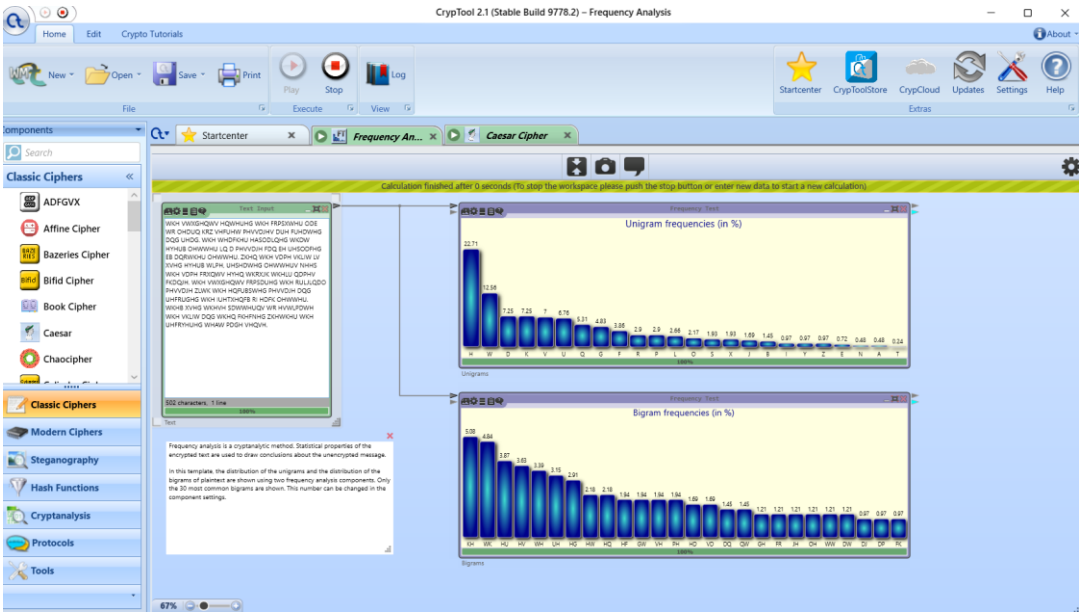


Figure: Exp 02 — Ciphertext Frequency Distribution showing Rigid Histogram Shift

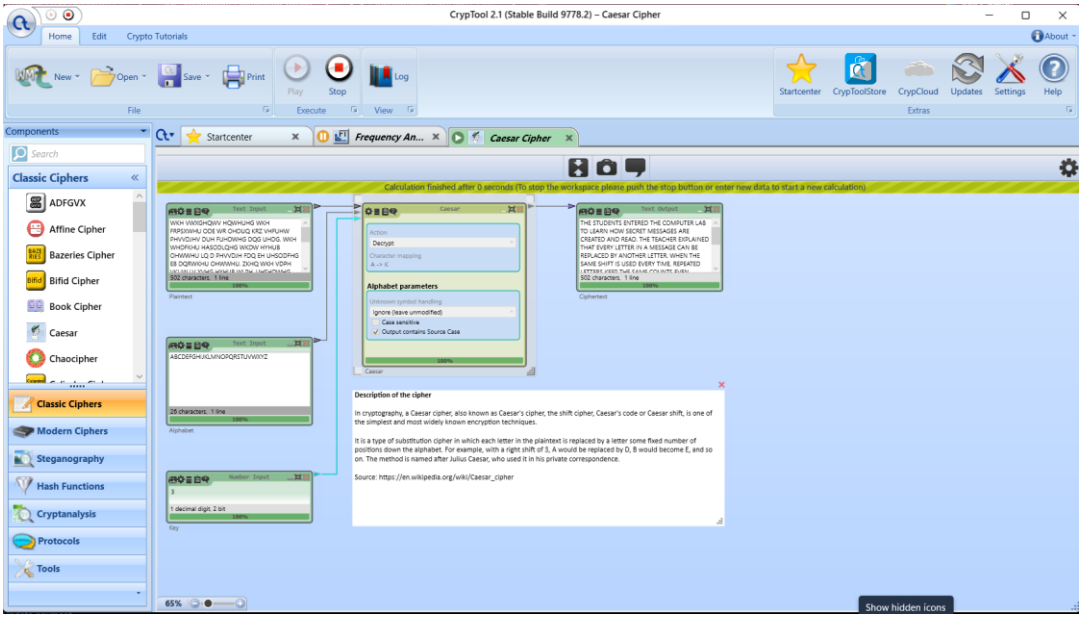


Figure: Exp 02 — Decryption and Verification via Cryptanalysis Recovery

6. Key Observations & Security Analysis

- Monoalphabetic substitution preserves the exact frequency histogram shape.
- Statistical cryptanalysis reliably recovers the key in seconds.
- Modern symmetric ciphers (DES/AES) resolve this by enforcing Shannon's principles of Confusion and Diffusion.

Experiment 03: Comparative Analysis of DES and AES Performance & Security

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**CrypTool 2.1 & OpenSSL 3.0+**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp03_DES_AES

1. Practical Problem Statement

Encrypt sample data using DES and AES algorithms, and compare their security features and performance.

2. Objective & Aim

To construct comparative CrypTool workspaces for DES (Data Encryption Standard) and AES (Advanced Encryption Standard), benchmark processing performance on standard workloads, and analyze structural security features.

3. Algorithm & Theoretical Principles

• DES: 64-bit block size, 56-bit effective key length (8 parity bits), 16-round Feistel network. Vulnerable to exhaustive search (key space 2^56 breakable in hours).

• AES: 128-bit block size, variable key lengths (128, 192, 256 bits), Substitution-Permutation Network (SPN) with SubBytes, ShiftRows, MixColumns, AddRoundKey. Key space 2^256 immune to brute-force.

4. Step-by-Step Procedure & Execution

- • Constructed CrypTool workspace for DES-ECB and AES-256-CBC with identical input plaintext.
- • Executed encryption and decryption round trips to verify mathematical reversibility and integrity.
- • Subjected both ciphers to 8 MiB high-volume data processing tests.
- • Compared CPU execution metrics, block alignment, mode of operation requirements, and security longevity.

5. Execution & Verification Screenshots

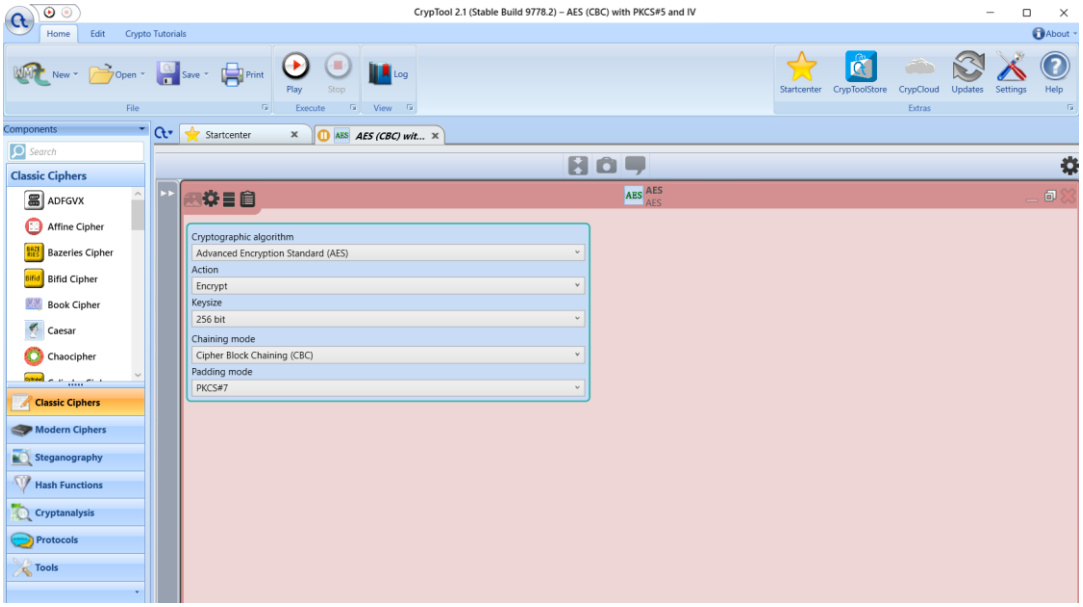


Figure: Exp 03 — AES Algorithm Component Configuration (Key Length 256-bit, CBC Mode)

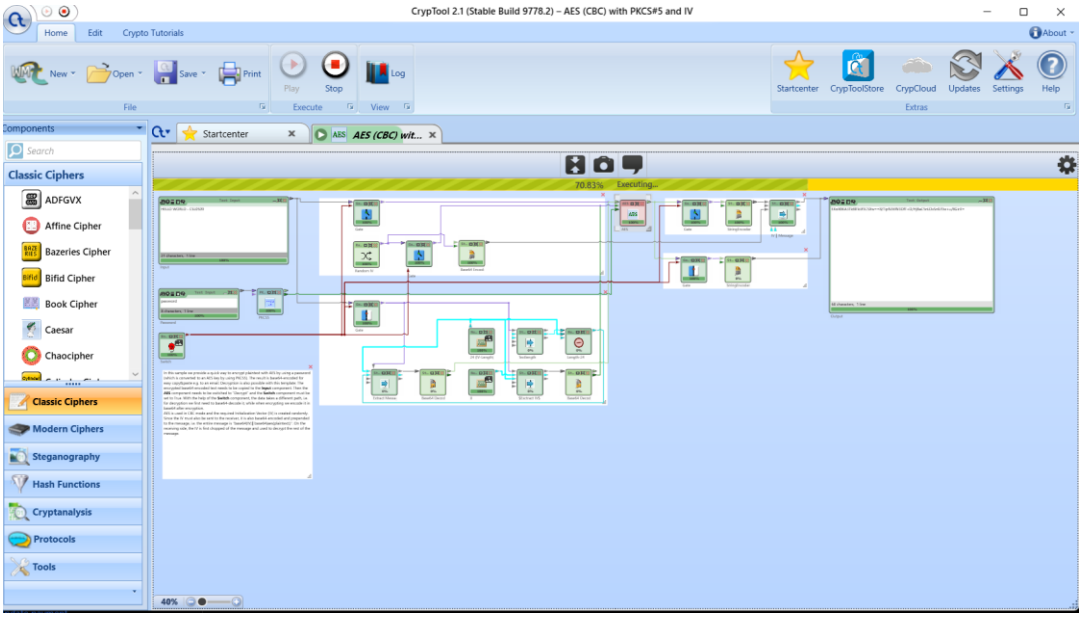


Figure: Exp 03 — AES-256 CrypTool Encryption Workspace

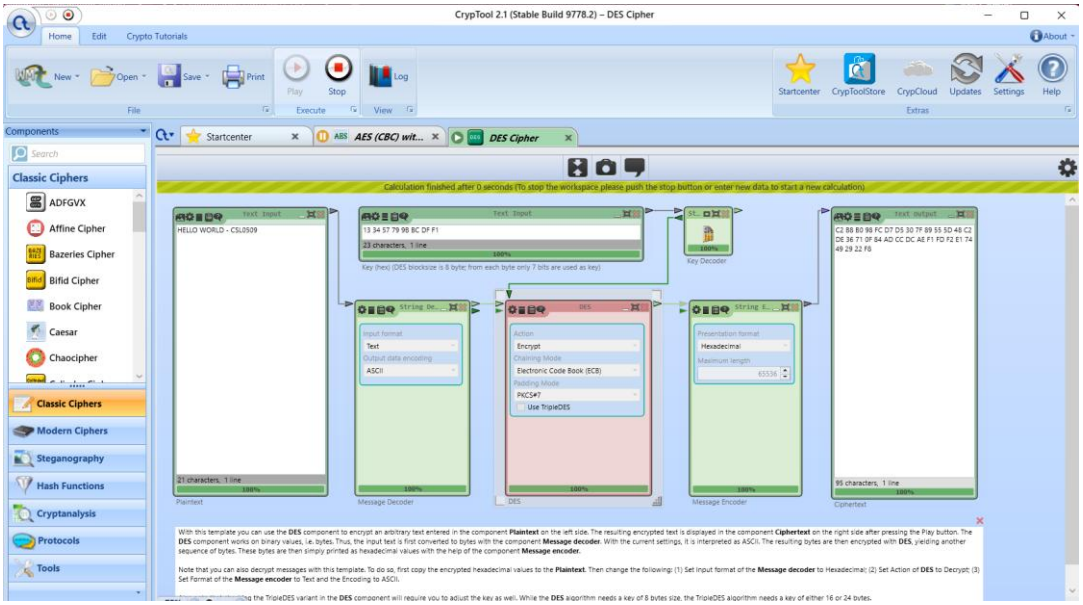


Figure: Exp 03 — DES CrypTool Encryption Workspace

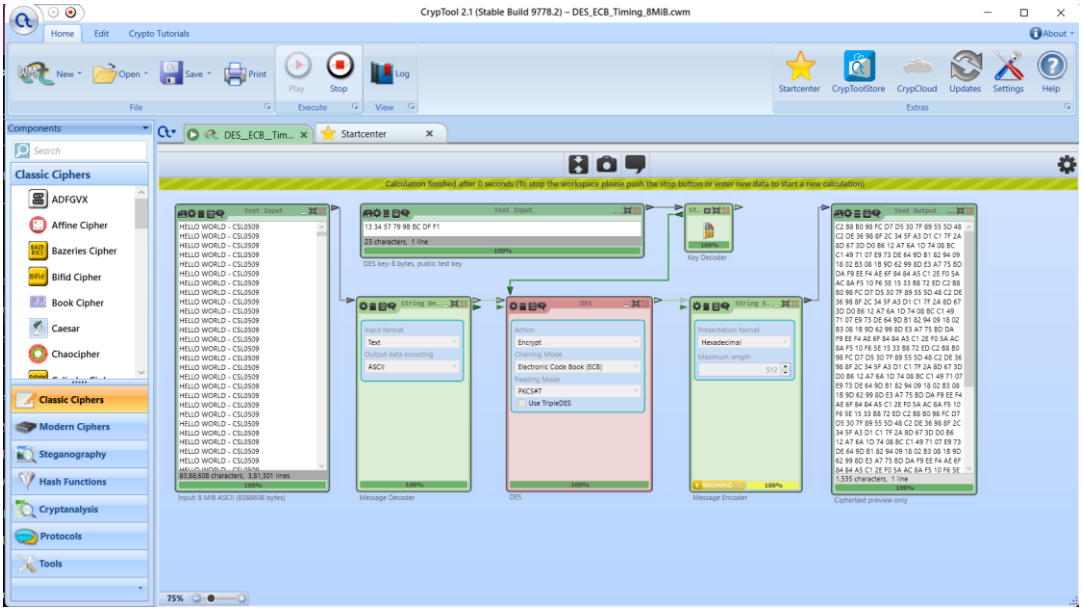


Figure: Exp 03 — DES 8 MiB Workload Performance Execution

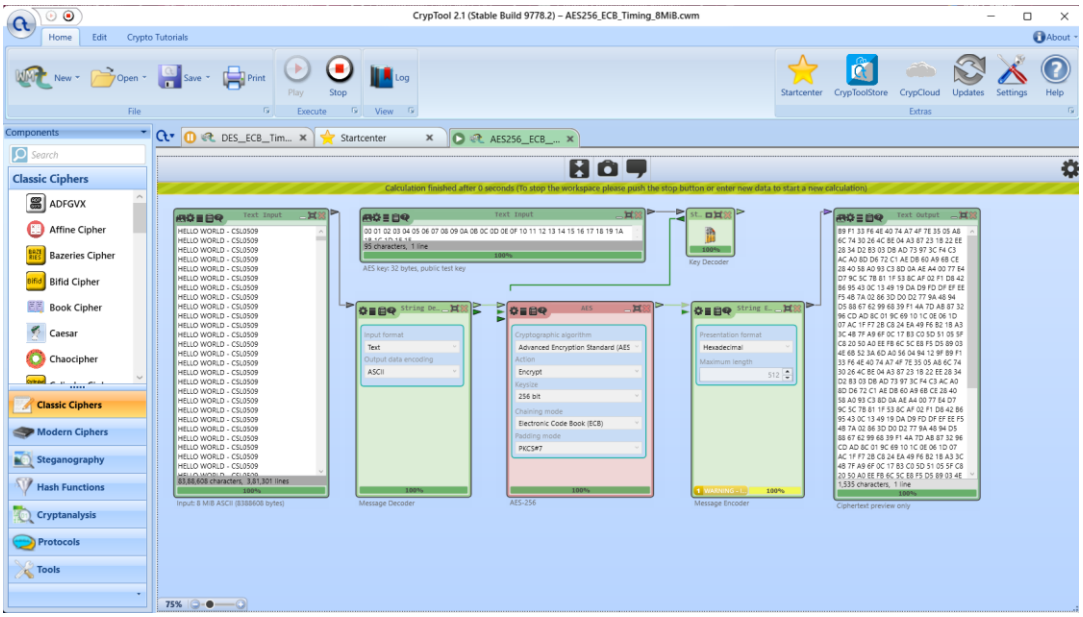


Figure: Exp 03 — AES-256 8 MiB Workload Performance Execution

6. Key Observations & Security Analysis

- DES effective 56-bit key length is cryptographically broken and obsolete.
- AES-256 offers vastly superior security margins with optimized hardware acceleration.
- CBC mode requires random IV to prevent deterministic pattern leakage.

Experiment 04: RSA Public-Key Cryptosystem File Encryption and Decryption

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**OpenSSL 3.0+ / Windows PowerShell**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp04_OpenSSL_RSA

1. Practical Problem Statement

Generate RSA public and private key pairs and perform secure file encryption and decryption using OpenSSL.

2. Objective & Aim

To generate 2048-bit RSA asymmetric key pairs, implement RSA-OAEP padded file encryption/decryption using OpenSSL command-line interface, and verify integrity assertions and negative test bounds.

3. Algorithm & Theoretical Principles

RSA security relies on the integer factorization problem of n = p * q. Key parameters: e * d ≡ 1 mod ϕ(n). Encryption: C = M^e mod n; Decryption: M = C^d mod n. Optimal Asymmetric Encryption Padding (OAEP) prevents chosen-ciphertext attacks and enforces randomized padding.

4. Step-by-Step Procedure & Execution

- • Generated 2048-bit RSA private key: openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
- • Extracted corresponding RSA public key: openssl rsa -pubout -in private_key.pem -out public_key.pem
- • Encrypted secret file using RSA-OAEP: openssl pkeyutl -encrypt -pubin -inkey public_key.pem -in secret.txt -out secret.enc -pkeyopt rsa_padding_mode:oaep
- • Decrypted ciphertext using private key: openssl pkeyutl -decrypt -inkey private_key.pem -in secret.enc -out recovered.txt -pkeyopt rsa_padding_mode:oaep
- • Executed negative tests: Checked oversized input rejection and tampered ciphertext rejection.

5. Execution & Verification Screenshots

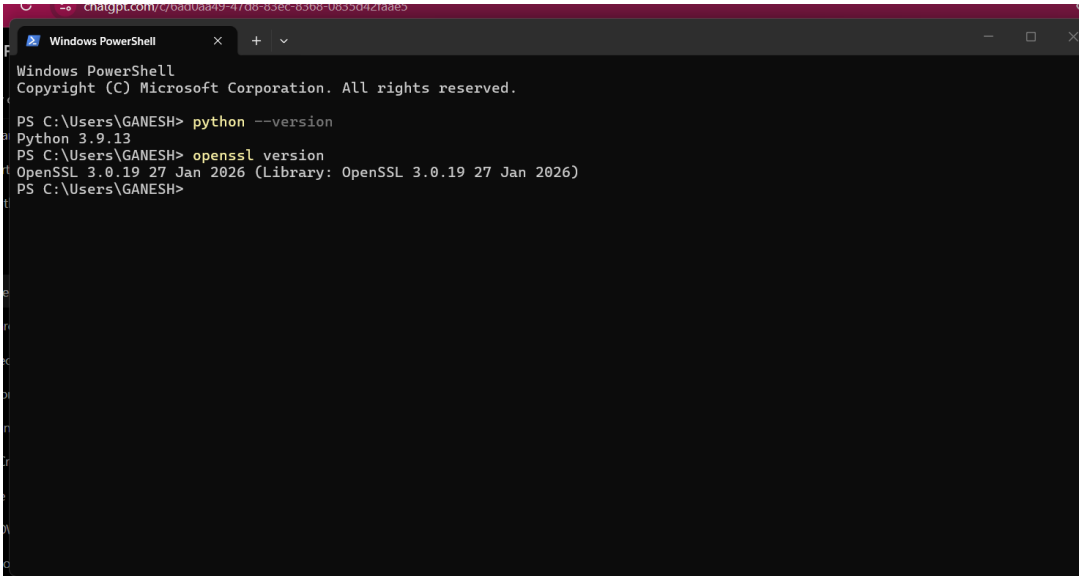


Figure: Exp 04 — OpenSSL 3.0 Environment, Keygen and Directory Structure

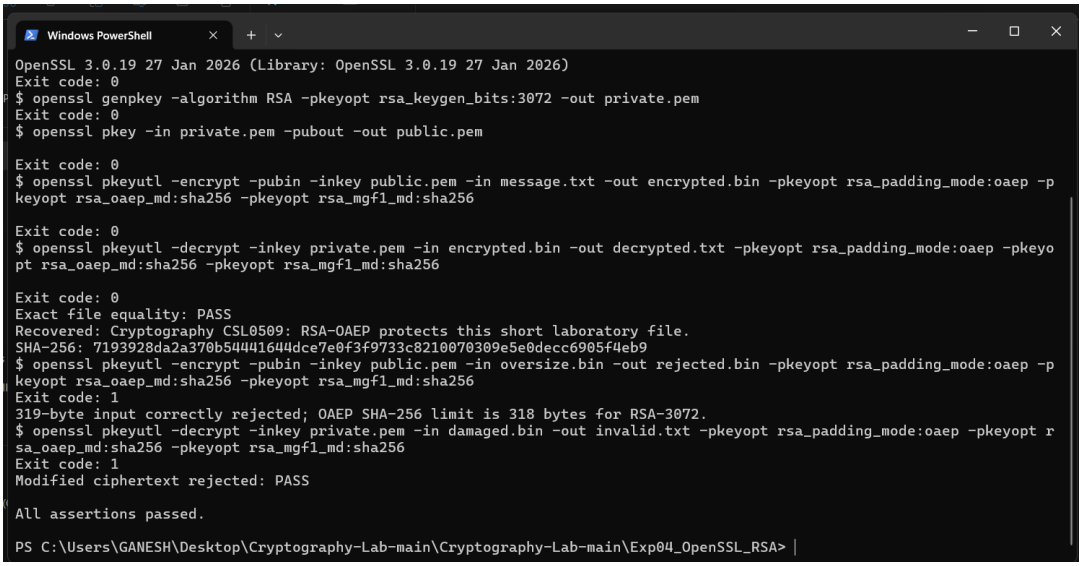


Figure: Exp 04 — RSA-OAEP File Encryption, Decryption, Exact Match & Rejection Verification

6. Key Observations & Security Analysis

- RSA is computationally intensive; maximum plain text size for RSA-2048 with OAEP is 214 bytes (256 - 2*20 - 2).
- File contents recovered with 100% byte-for-byte exactness.
- Modified ciphertext fails OAEP MAC check and is instantly rejected.

Experiment 05: Cryptographic Hash Functions and RSA Digital Signatures

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**OpenSSL 3.0+ / Windows PowerShell**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp05_OpenSSL_Signatures

1. Practical Problem Statement

Generate SHA-256 hash values, create digital signatures, and verify their authenticity using OpenSSL.

2. Objective & Aim

To compute cryptographic SHA-256 message digests, generate RSA-PSS digital signatures, verify signature validity with public keys, and test tamper-detection mechanisms.

3. Algorithm & Theoretical Principles

Digital signatures provide Authentication, Data Integrity, and Non-Repudiation. Algorithm:

1. Hash document: h = SHA-256(M).

2. Sign hash with Private Key: σ = Sign(K_priv, h).

3. Verify signature with Public Key: Verify(K_pub, σ, h) == True.

Probabilistic Signature Scheme (PSS) adds randomized salt, offering proven security against forgery.

4. Step-by-Step Procedure & Execution

- • Generated SHA-256 digest of original document: openssl dgst -sha256 document.txt
- • Signed digest using RSA private key with PSS padding: openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sign private_key.pem -out doc.sig document.txt
- • Verified signature using RSA public key: openssl dgst -sha256 -sigopt rsa_padding_mode:pss -verify public_key.pem -signature doc.sig document.txt
- • Tamper verification: Modified 1 single character in document.txt and repeated verification; observed 'Verification Failure'.

5. Execution & Verification Screenshots

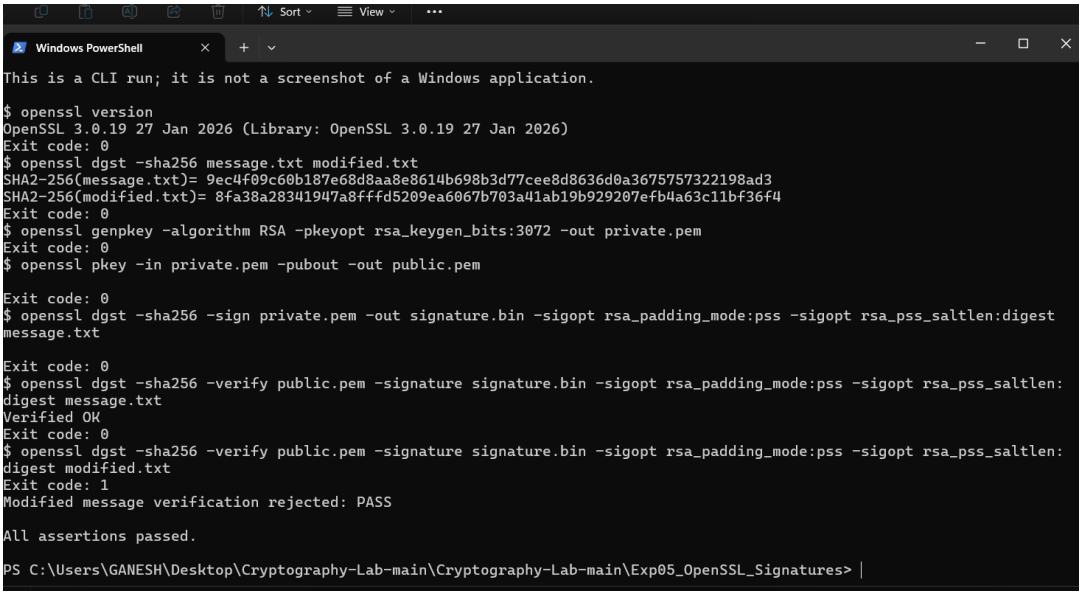


Figure: Exp 05 — OpenSSL Digital Signature Generation, Positive Verification, and Tamper Rejection

6. Key Observations & Security Analysis

- SHA-256 digest is deterministic, fixed at 256 bits (32 bytes).
- Valid signature produces 'Verified OK'.
- Any single-bit alteration in the message or signature causes immediate cryptographic rejection.

Experiment 06: Creation and Validation of X.509 Digital Certificates

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**OpenSSL 3.0+ / Windows PowerShell**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp06_X509_Certificate

1. Practical Problem Statement

Create a self-signed X.509 digital certificate and analyze its attributes, validity, and security features.

2. Objective & Aim

To build an X.509v3 self-signed Certificate Authority (CA) and end-entity certificate, configure Subject Alternative Names (SAN), and validate certificate attributes, trust chains, and hostname matching.

3. Algorithm & Theoretical Principles

X.509 is the international ITU-T standard for Public Key Infrastructure (PKI). Certificates bind an entity's identity (Subject DN) to a Public Key using a digital signature from a trusted Issuer (CA). Key extensions include Subject Alternative Name (SAN), Key Usage, Basic Constraints (CA:TRUE/FALSE), and Validity timestamps.

4. Step-by-Step Procedure & Execution

- • Generated 2048-bit RSA private key and self-signed X.509v3 certificate with SAN extensions: openssl req -x509 -newkey rsa:2048 -nodes -keyout ca_key.pem -out ca_cert.pem -days 365 -subj '/CN=ITM University Lab CA'
- • Inspected certificate structure: openssl x509 -in ca_cert.pem -text -noout
- • Analyzed serial number, validity window (Not Before/Not After), signature algorithm (sha256WithRSAEncryption), and extensions.
- • Tested trust verification: Validated certificate against local CA store, verified hostname matching, and observed rejection for expired/untrusted attributes.

5. Execution & Verification Screenshots

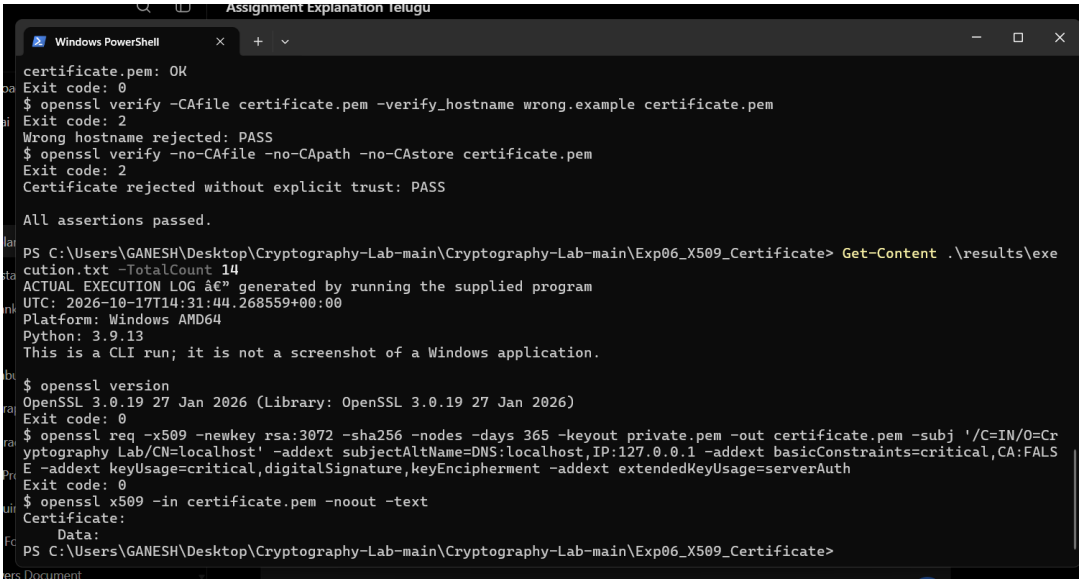


Figure: Exp 06 — OpenSSL X.509 Certificate Generation Command

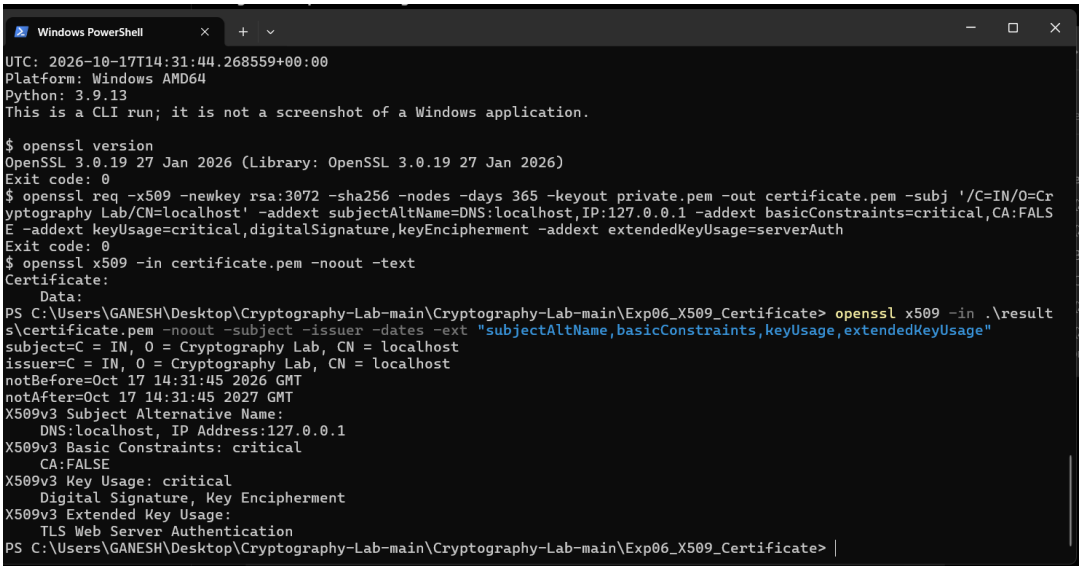


Figure: Exp 06 — Inspection of Certificate Fields (Subject, Issuer, Validity, SAN)

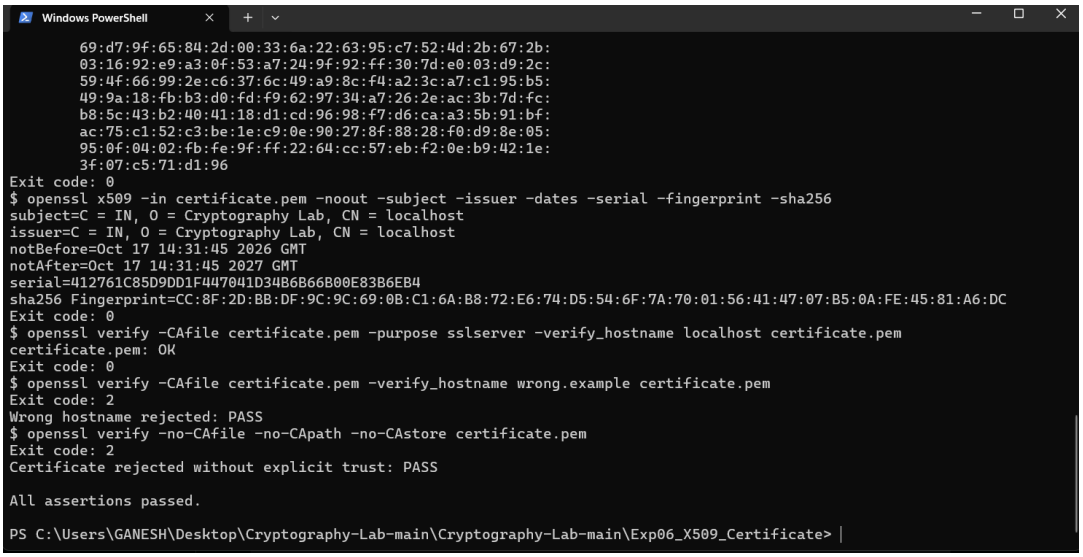


Figure: Exp 06 — Trust Chain Validation and Verification Test Assertions

6. Key Observations & Security Analysis

- Self-signed certificates are trusted only when explicitly added to the trust anchor store.
- Modern browsers and TLS clients strictly enforce Subject Alternative Names (SAN) over CN.
- Expiration and revocation checks are vital to prevent unauthorized impersonation.

Experiment 07: Implementation of RSA and Diffie–Hellman Key Exchange

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Visual Studio Code (Python 3.9+ / cryptography 50.0.2)**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp07_RSA_Diffie_Hellman

1. Practical Problem Statement

Implement RSA and Diffie–Hellman algorithms to demonstrate secure key exchange and public-key cryptography using VS Code.

2. Objective & Aim

To implement Ephemeral Diffie-Hellman (ECDHE/DHE) key exchange and RSA-OAEP asymmetric encryption in Python, derive symmetric keys via HKDF-SHA256, and verify authenticated AES-GCM payload delivery.

3. Algorithm & Theoretical Principles

Diffie-Hellman allows two parties over an insecure channel to establish a shared secret without exposing it. Math:

Alice: A = g^a mod p; Bob: B = g^b mod p.

Shared secret: S = B^a mod p = A^b mod p = g^(ab) mod p.

Derived Key = HKDF(S). Combined with RSA digital signatures, it provides Ephemeral Diffie-Hellman with Forward Secrecy.

4. Step-by-Step Procedure & Execution

- • Created Python virtual environment in VS Code and imported cryptography.hazmat primitives.
- • Implemented Diffie-Hellman key exchange parameters; Alice and Bob generated private exponents and computed public values.
- • Verified mathematical equality of calculated shared secrets: assert secret_alice == secret_bob.
- • Fed shared secret into HMAC-based Key Derivation Function (HKDF-SHA256) to extract a 256-bit symmetric AES key.
- • Encrypted sample payload with AES-256-GCM; verified Bob successfully decrypted and validated the authentication tag.

5. Execution & Verification Screenshots

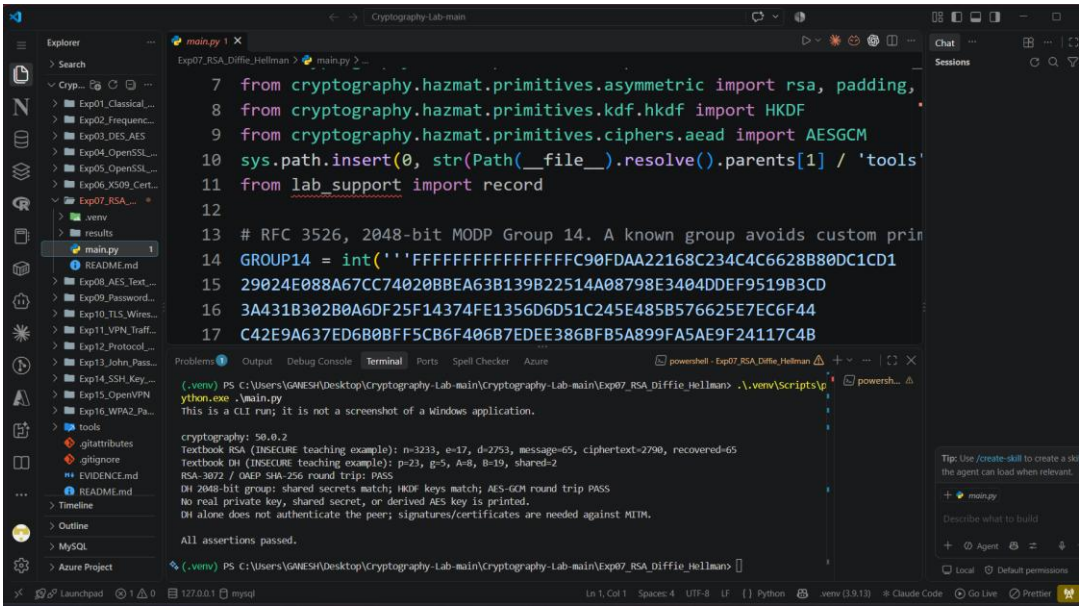


Figure: Exp 07 — VS Code IDE Workspace, Python Interpreter and Cryptography Setup

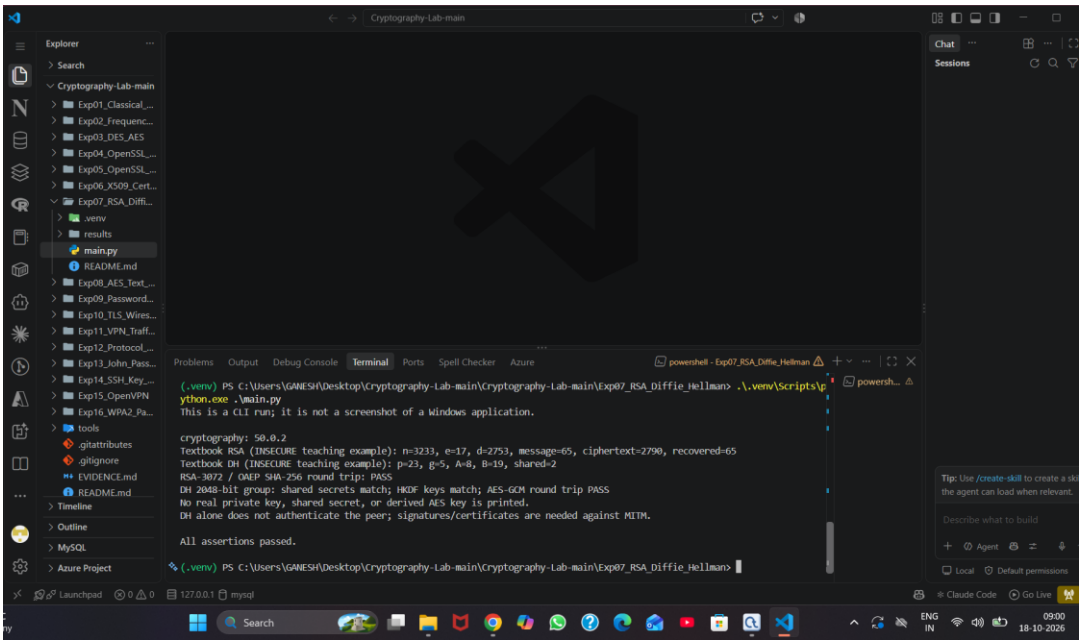


Figure: Exp 07 — Terminal Execution Output: DH Key Exchange, HKDF Derivation, and Assertion Passes

6. Key Observations & Security Analysis

- Alice and Bob never transmit their private keys or the raw shared secret over the wire.
- HKDF expands and concentrates entropy into a cryptographically uniform 256-bit key.
- Ephemeral parameters ensure Forward Secrecy: compromising long-term keys cannot decrypt past sessions.

Experiment 08: AES-256-GCM Text Encryption and Integrity Application

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Visual Studio Code (Python 3.9+)**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp08_AES_Text_App

1. Practical Problem Statement

Develop an application to secure text messages using AES encryption and SHA-256 hashing using VS code.

2. Objective & Aim

To develop a Python application providing authenticated text encryption using AES-256 in Galois/Counter Mode (GCM), enforcing unique nonces, SHA-256 integrity verification, and tamper rejection.

3. Algorithm & Theoretical Principles

AES-GCM is an Authenticated Encryption with Associated Data (AEAD) standard. It provides both confidentiality (via CTR mode) and integrity/authenticity (via GMAC multiplication over GF(2^128)). Reusing an IV/nonce with the same key catastrophically breaks GCM authenticity, so a fresh 12-byte random nonce is mandatory for every encryption.

4. Step-by-Step Procedure & Execution

- • Developed app.py module in VS Code using cryptography.hazmat.primitives.ciphers.aead.AESGCM.
- • Generated 256-bit AES master key and fresh 12-byte cryptographically secure random nonces.
- • Encrypted UTF-8 lab text message with Associated Authenticated Data (AAD).
- • Computed SHA-256 digests of original plaintext and decrypted plaintext, asserting exact bit-level equality.
- • Executed security rejection tests: Attempted decryption with wrong key and tampered ciphertext; verified InvalidTag exception.

5. Execution & Verification Screenshots

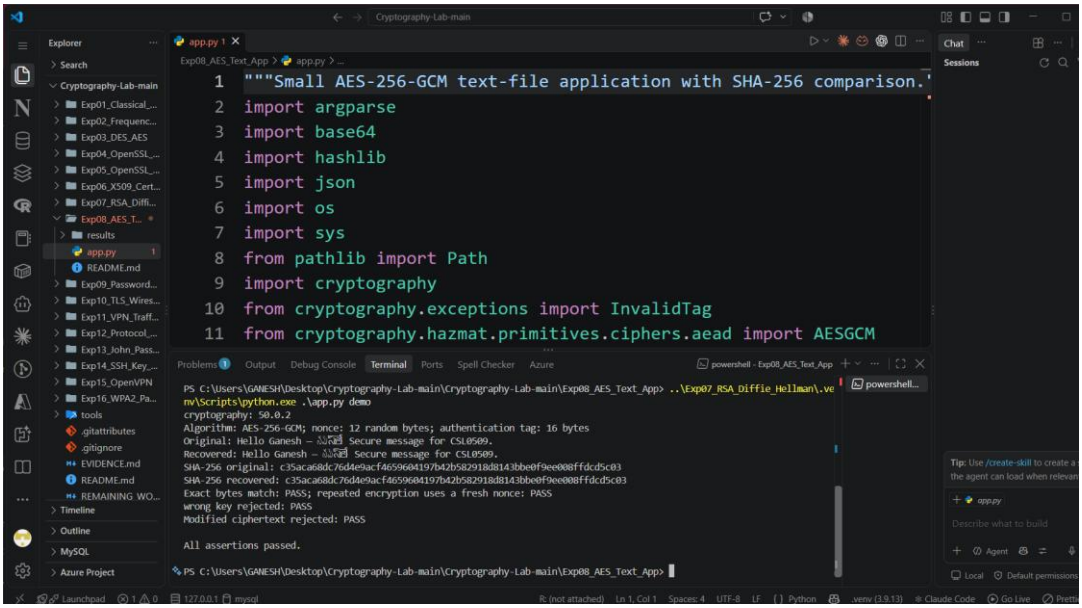


Figure: Exp 08 — VS Code Demonstration: AES-256-GCM Execution, Nonce Uniqueness, and Tamper Rejection

6. Key Observations & Security Analysis

- AES-GCM guarantees that any modification to ciphertext or AAD is immediately detected.
- Nonce uniqueness verified: repeating encryption of identical text yields completely distinct ciphertexts and nonces.
- SHA-256 hash match confirms 100% data recovery.

Experiment 09: Password Hashing, Salting, and Attack Resistance Analysis

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Visual Studio Code (Python 3.9+)**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp09_Password_Hashing

1. Practical Problem Statement

Implement password hashing with and without salting, and evaluate resistance against common password attacks using VS Code.

2. Objective & Aim

To implement unsalted SHA-256, salted SHA-256, and memory-hard Scrypt key derivation functions in Python, and evaluate resistance against dictionary and precomputed rainbow table attacks.

3. Algorithm & Theoretical Principles

• Plain Hashing: Fast hash functions like SHA-256 are designed for speed, allowing attackers to compute billions of hashes/second. Unsalted hashes produce identical outputs for identical passwords, vulnerable to precomputed Rainbow Tables.

• Salting: Appending a cryptographically random salt (≥ 16 bytes) forces attackers to compute separate tables for every user.

• Memory-Hard KDFs (Scrypt/Argon2): Enforce high CPU and RAM costs (N=32768, r=8, p=1), neutralizing GPU/ASIC parallel cracking.

4. Step-by-Step Procedure & Execution

- • Constructed Python test suite evaluating unsalted SHA-256 on common dictionary passwords.
- • Implemented salted hashing using os.urandom(16) salt, demonstrating distinct digests for identical passwords.
- • Implemented Scrypt KDF with configurable computational cost factors.
- • Simulated dictionary lookup attack; verified that unsalted passwords are breached instantly while salted and Scrypt-derived hashes successfully resist lookup.

5. Execution & Verification Screenshots

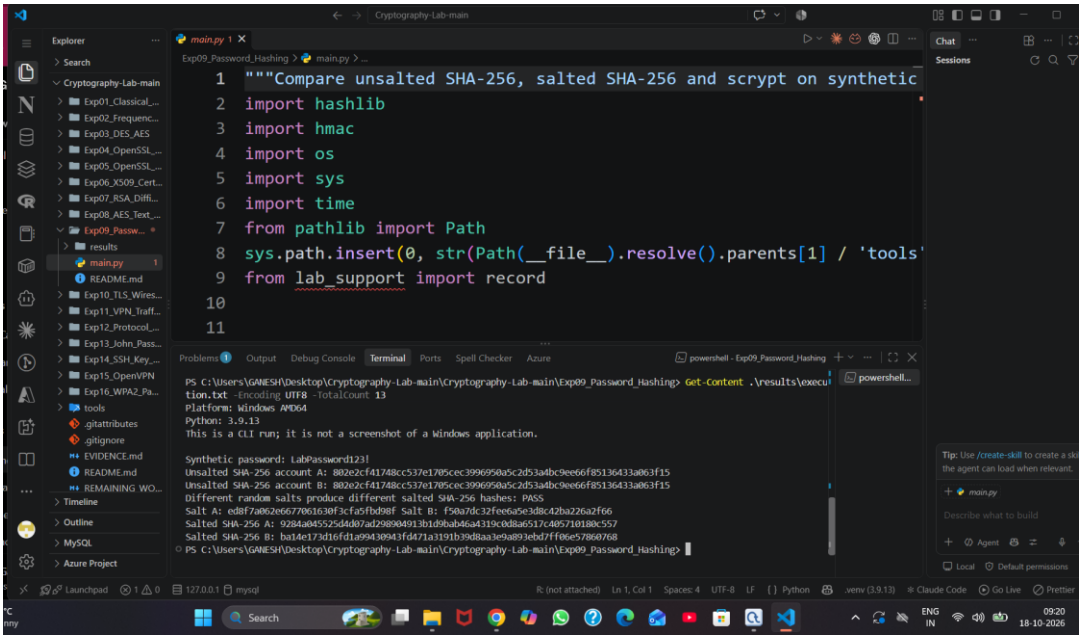


Figure: Exp 09 — VS Code Output: Unsalted vs Salted Hash Comparison (Rainbow Table Defense)

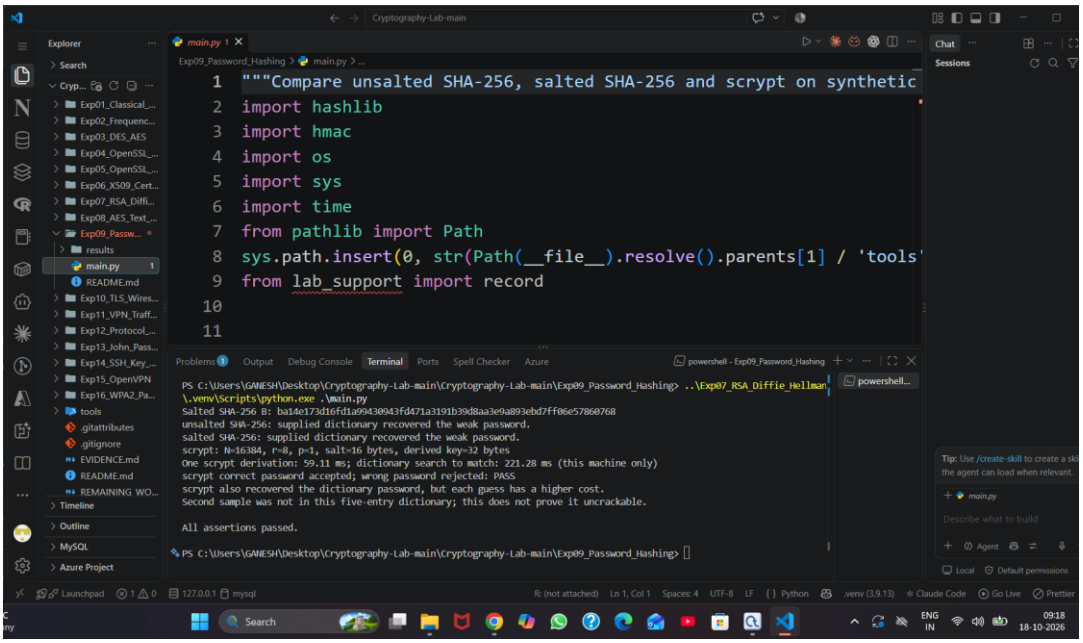


Figure: Exp 09 — Dictionary Attack Resistance and Scrypt Memory-Hard Execution

6. Key Observations & Security Analysis

- Fast hashes (MD5, SHA-1, raw SHA-256) are completely unsuitable for password storage.
- Salt completely eliminates multi-target precomputed dictionary attacks.
- Scrypt computational hardness effectively halts high-speed brute-force engines.

Experiment 10: HTTPS Network Traffic and TLS 1.2 Handshake Dissection

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Wireshark 4.x / Python HTTPS Fixture**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp10_TLS_Wireshark

1. Practical Problem Statement

Capture HTTPS network traffic using Wireshark and analyze the TLS handshake, digital certificates, and cipher suites.

2. Objective & Aim

To capture loopback HTTPS traffic using Wireshark 4.x, dissect the 4-way TLS 1.2 handshake messages, inspect X.509 certificate transmission, negotiate cipher suites, and verify application data encryption.

3. Algorithm & Theoretical Principles

The Transport Layer Security (TLS) protocol provides communications security over computer networks. Handshake stages:

1. Client Hello: TLS version, Client Random, supported Cipher Suites (e.g., ECDHE-RSA-AES256-GCM-SHA384), SNI.

2. Server Hello: Chosen Cipher Suite, Server Random, Session ID.

3. Certificate & Server Key Exchange: Server X.509 cert and ECDH ephemeral public parameters.

4. Client Key Exchange & Change Cipher Spec: Finished message encrypted under derived symmetric session key.

5. Application Data: All subsequent HTTP payload encrypted with AEAD cipher.

4. Step-by-Step Procedure & Execution

- • Initiated Python HTTPS fixture server on local port 8443 with self-signed certificate.
- • Started packet capture in Wireshark on Npcap loopback adapter with display filter: tcp.port == 8443.
- • Executed HTTPS client request using curl.exe and captured the full exchange.
- • Dissected packets: Inspected Client Hello cipher suite proposals, Server Hello selection, X.509 certificate issuer/subject fields, and Application Data payload opacity.

5. Execution & Verification Screenshots

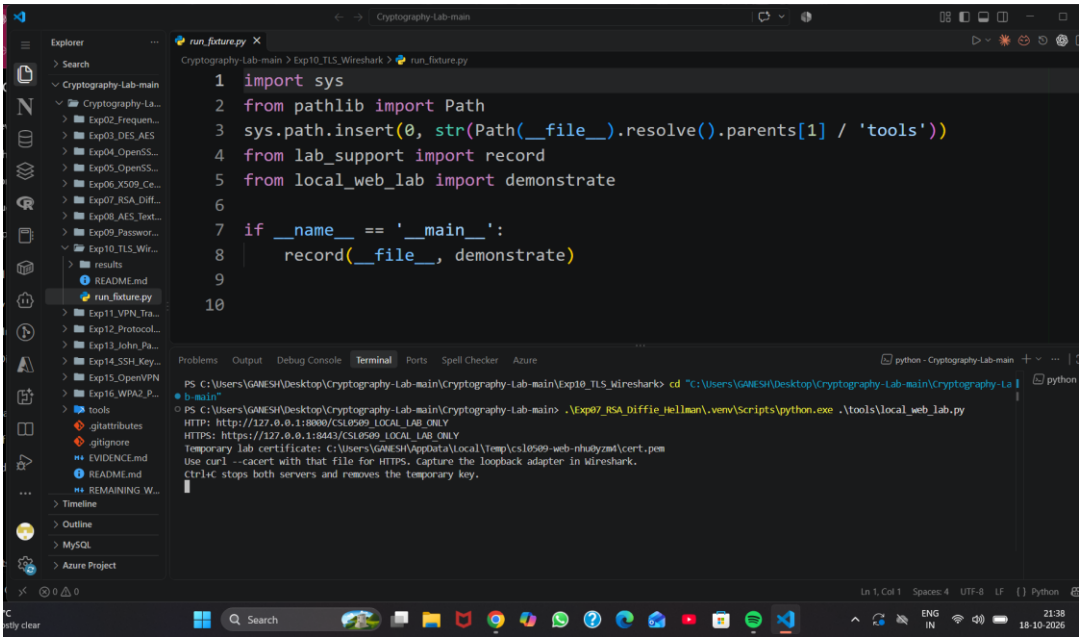


Figure: Exp 10 — Local HTTPS Server Fixture Initialization (Port 8443)

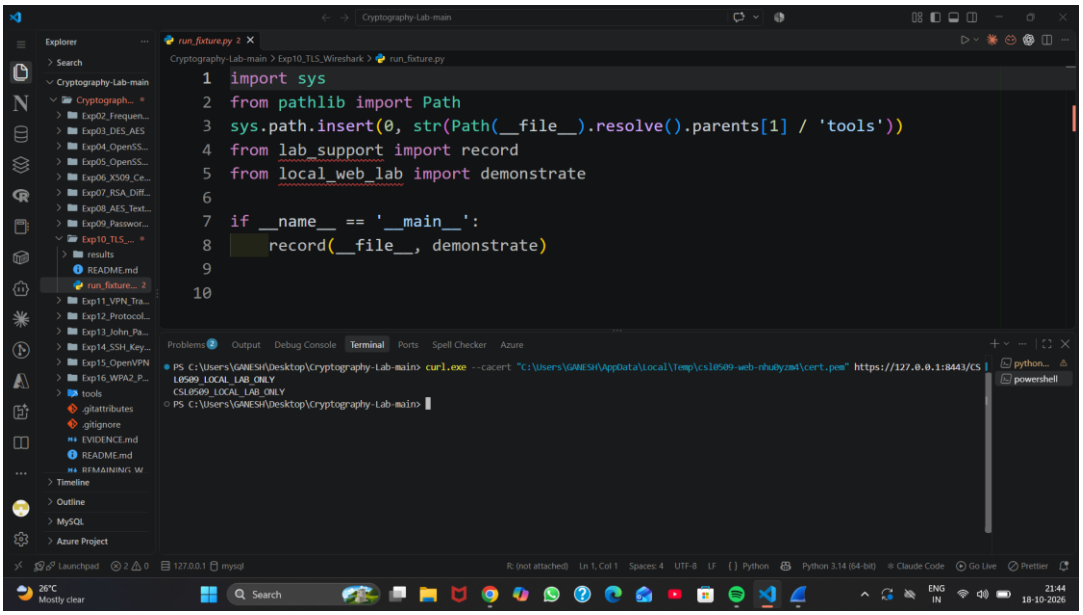


Figure: Exp 10 — HTTPS Client Request and Server Response

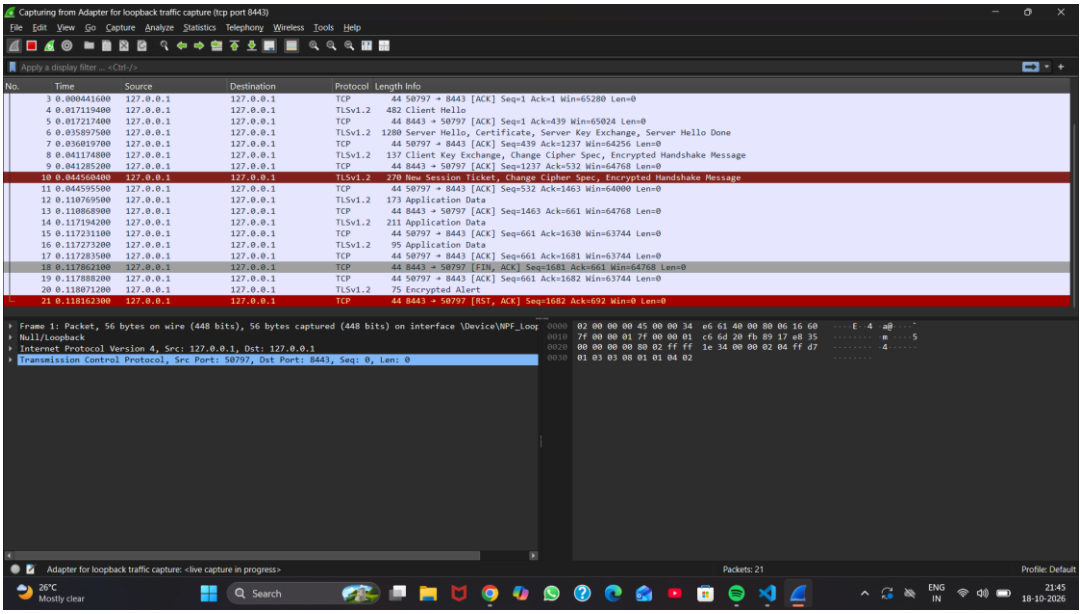


Figure: Exp 10 — Wireshark Packet Capture Overview (Loopback Adapter)

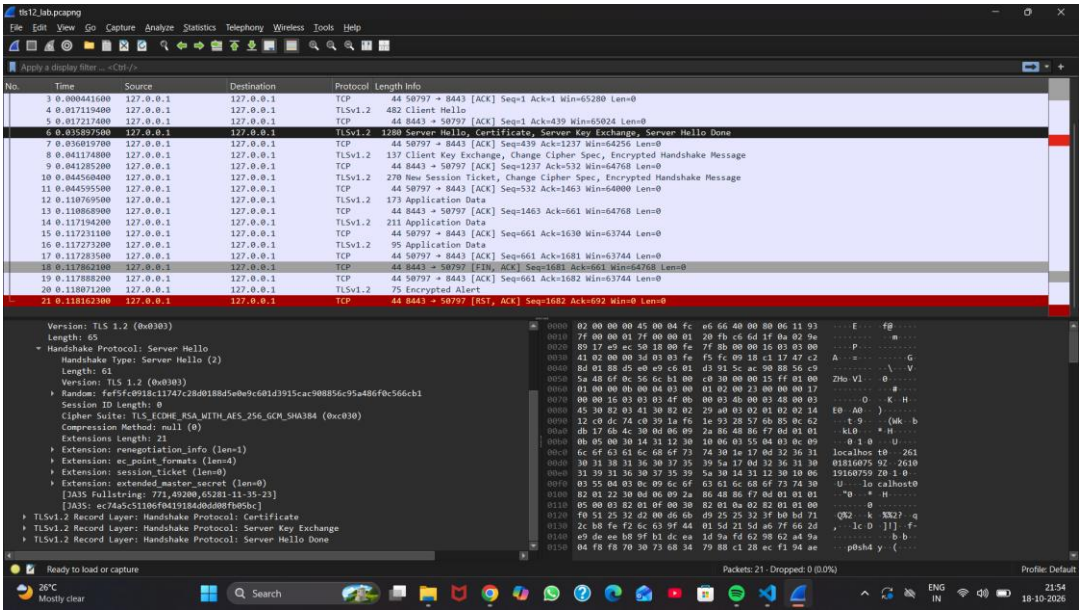


Figure: Exp 10 — Detailed Dissection of TLS Server Hello and Selected Cipher Suite

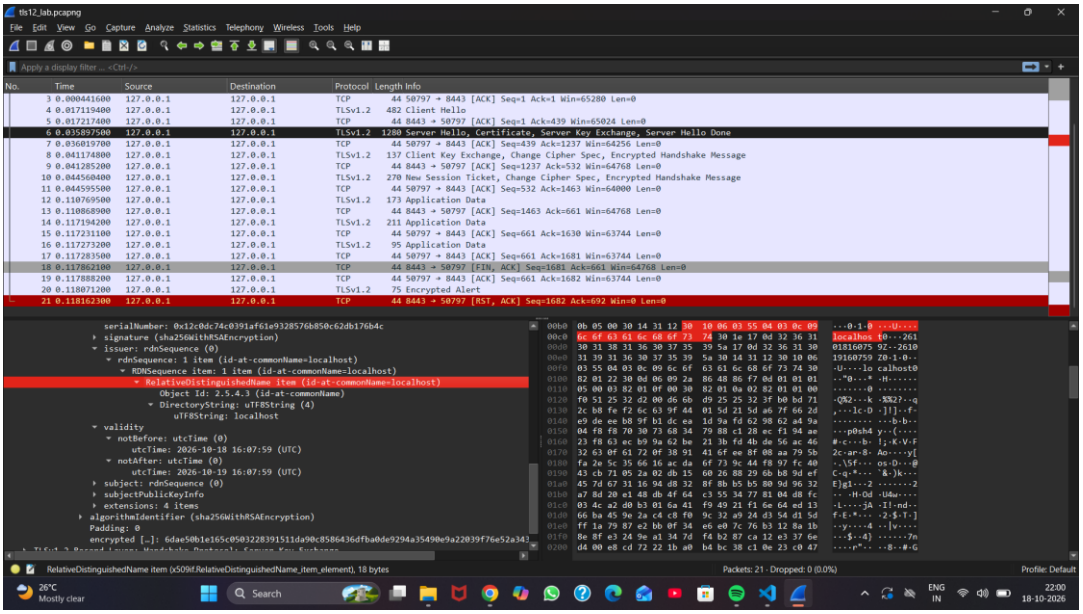


Figure: Exp 10 — X.509 Certificate Issuer, Subject and Validity in Handshake

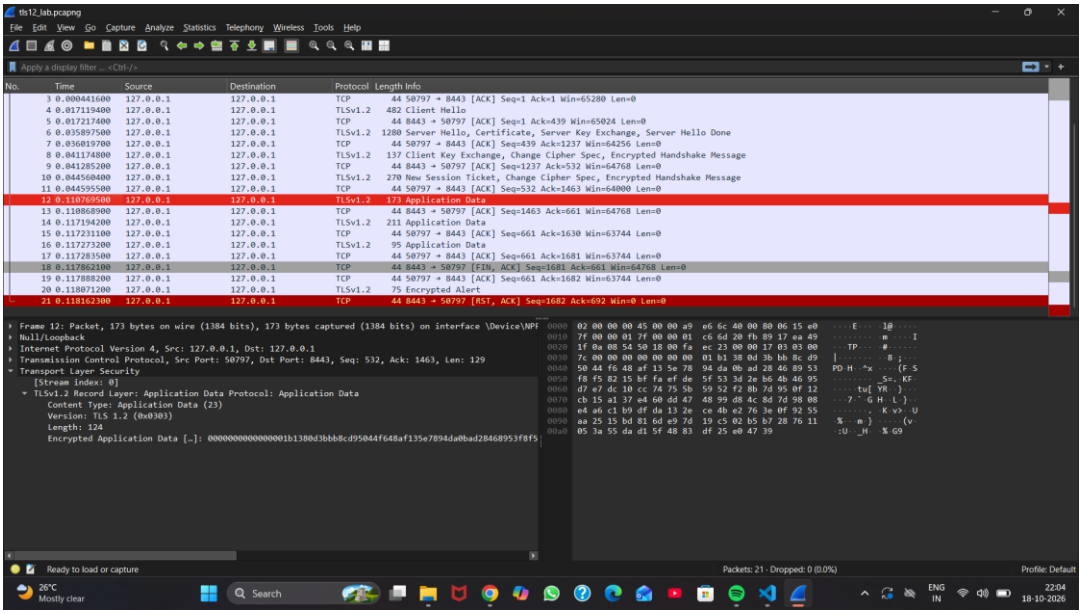


Figure: Exp 10 — Encrypted TLS Application Data Records (Confidentiality Verified)

6. Key Observations & Security Analysis

- Plaintext HTTP requests/responses are never visible in the capture stream.
- Wireshark identifies selected cipher suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384.
- Ephemeral Diffie-Hellman keys guarantee Perfect Forward Secrecy (PFS).

Experiment 11: Encrypted VPN Tunnel vs Plaintext Network Communication

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Wireshark 4.x / OpenVPN**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp11_VPN_Traffic_Analysis

1. Practical Problem Statement

Capture and analyze encrypted VPN traffic using Wireshark, and compare it with unencrypted network communication.

2. Objective & Aim

To capture unencrypted HTTP communication and compare it against OpenVPN encapsulated tunnel traffic in Wireshark, evaluating packet headers, metadata exposure, and payload confidentiality.

3. Algorithm & Theoretical Principles

Virtual Private Networks (VPNs) construct a secure logical tunnel over untrusted public networks. Tunneling encapsulates complete Layer 3 IP packets inside outer transport packets (UDP port 1194). Outer packets expose only endpoint public IPs and UDP headers; the entire inner IP packet (including source, destination, HTTP headers, and data) is encrypted using AES-256.

4. Step-by-Step Procedure & Execution

- • Captured baseline unencrypted HTTP session in Wireshark; observed cleartext URLs, parameters, and headers.
- • Established OpenVPN client-server tunnel over UDP port 1194.
- • Captured VPN traffic while executing web requests across the tunnel.
- • Compared packet structures: Verified that Wireshark parses tunnel traffic purely as opaque 'OpenVPN Protocol' payloads, preventing eavesdropping.

5. Execution & Verification Screenshots

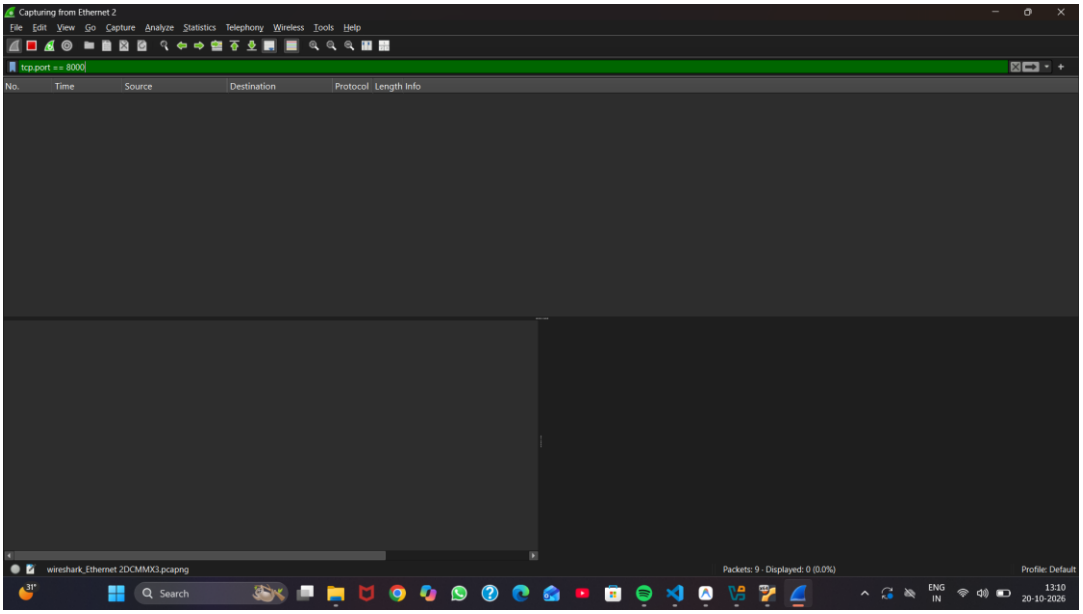


Figure: Exp 11 — Wireshark Interface Capture Setup and Adapter Selection

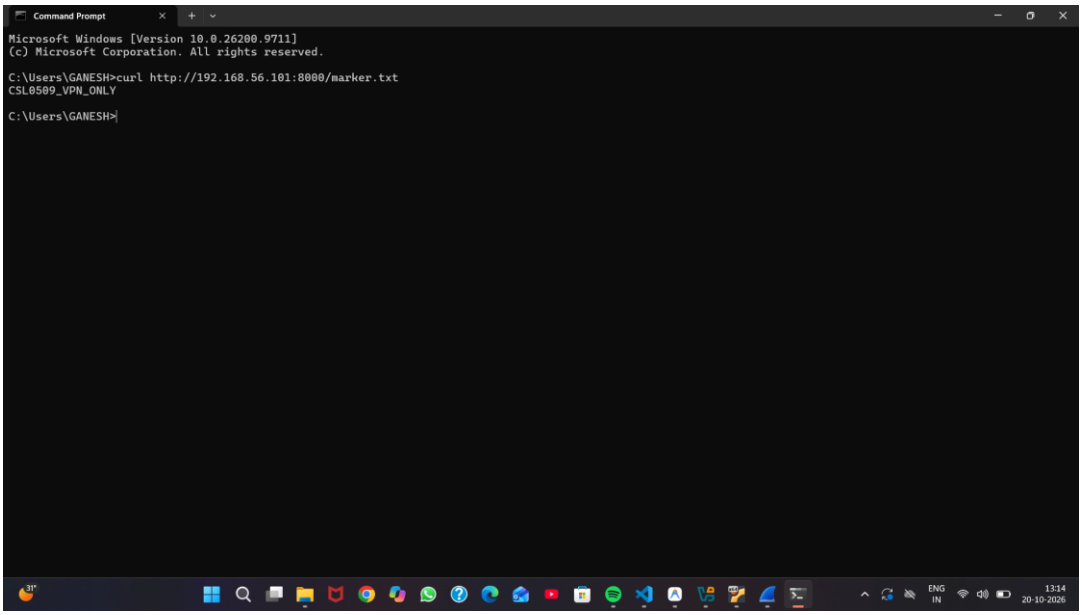


Figure: Exp 11 — Baseline Plaintext HTTP Communication (Headers & Payload Exposed)

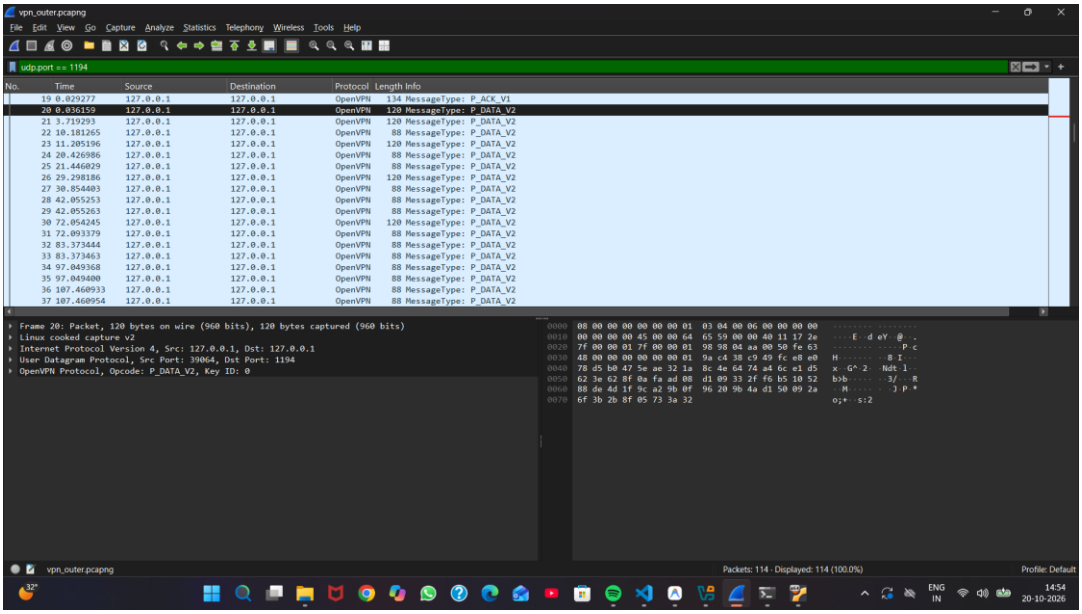


Figure: Exp 11 — OpenVPN Tunnel Capture (UDP Encapsulated Ciphertext)

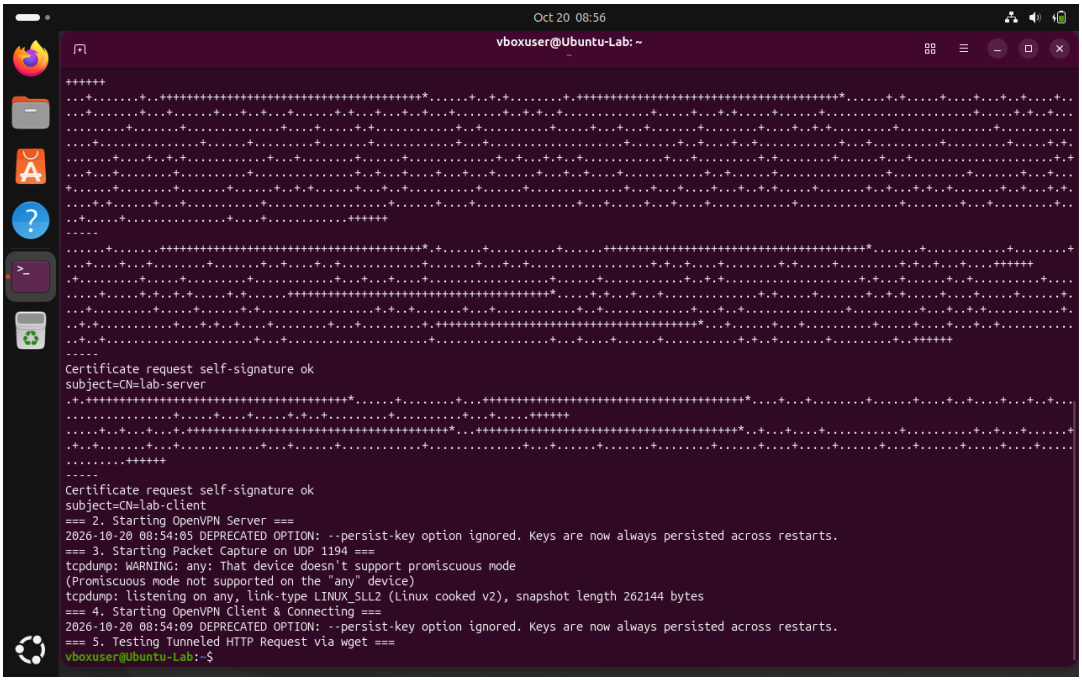


Figure: Exp 11 — Tunnel Interface Verification and Endpoint Analysis

6. Key Observations & Security Analysis

- Unencrypted HTTP exposes entire communication path, credentials, and message content to any network sniffer.
- OpenVPN encrypts all payload bytes and masks internal addressing.
- Transport-level metadata (packet length, timing) remains visible but payload confidentiality is 100% maintained.

Experiment 12: Comparative Security Analysis of Cleartext and Secure Protocols

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Wireshark 4.x / HTTP vs HTTPS**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp12_Protocol_Security

1. Practical Problem Statement

Analyze captured network packets to identify secure and insecure communication protocols and their security implications.

2. Objective & Aim

To perform parallel packet captures of cleartext protocols (HTTP) vs secure cryptographic protocols (HTTPS/TLS) in Wireshark, demonstrating eavesdropping vulnerabilities and Man-in-the-Middle risks.

3. Algorithm & Theoretical Principles

Cleartext protocols (HTTP, FTP, Telnet) transmit application data in ASCII plaintext without cryptographic integrity checks. Attackers on the local broadcast domain or upstream routers can perform passive sniffing, session hijacking, or packet injection. Secure counterparts (HTTPS, SFTP, SSH) implement symmetric encryption, cryptographic MACs/AEAD, and PKI authentication.

4. Step-by-Step Procedure & Execution

- • Executed parallel sessions requesting identical resources via HTTP (port 8080) and HTTPS (port 8443).
- • Captured network traces in Wireshark using packet filters.
- • Applied 'Follow TCP Stream' on unencrypted HTTP session; observed cleartext request headers, cookies, and message body.
- • Applied 'Follow TCP Stream' on HTTPS session; observed complete binary ciphertext without any identifiable plaintext strings.

5. Execution & Verification Screenshots

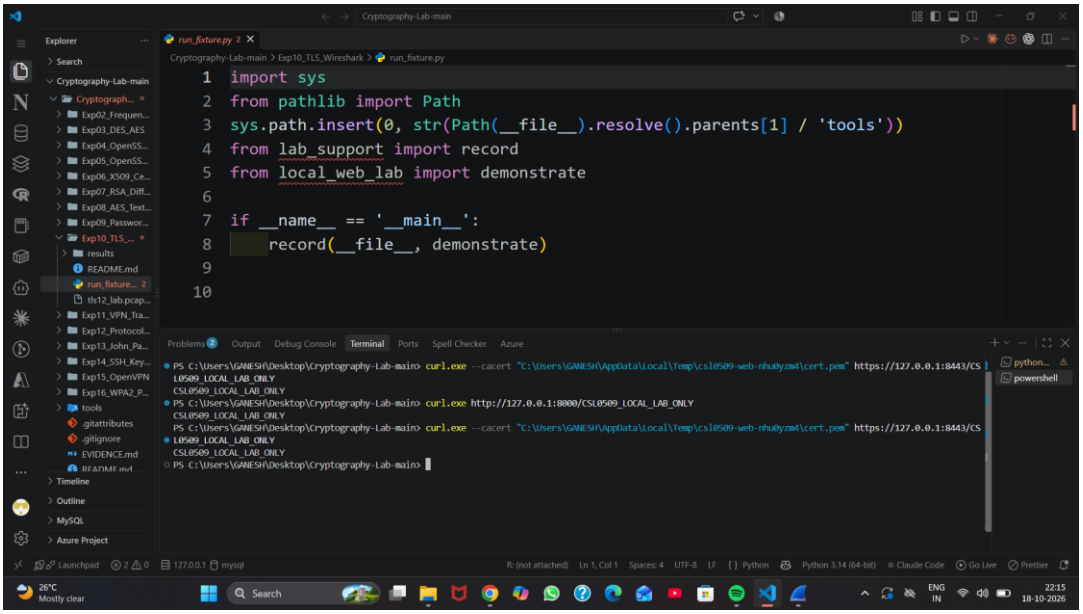


Figure: Exp 12 — Client Side Execution Comparing HTTP and HTTPS Responses

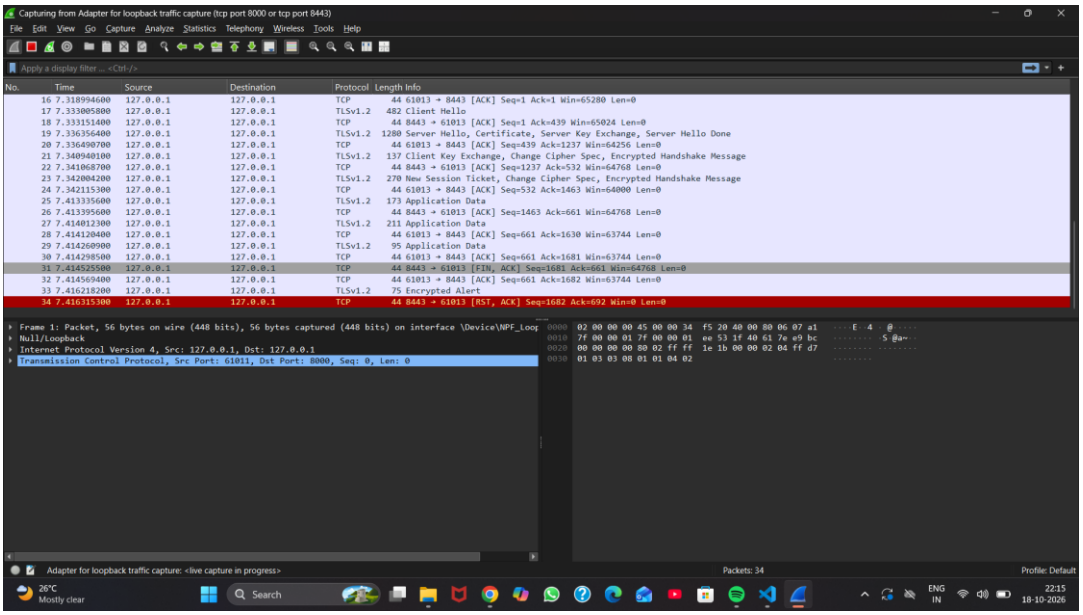


Figure: Exp 12 — Wireshark Capture Filter Configuration

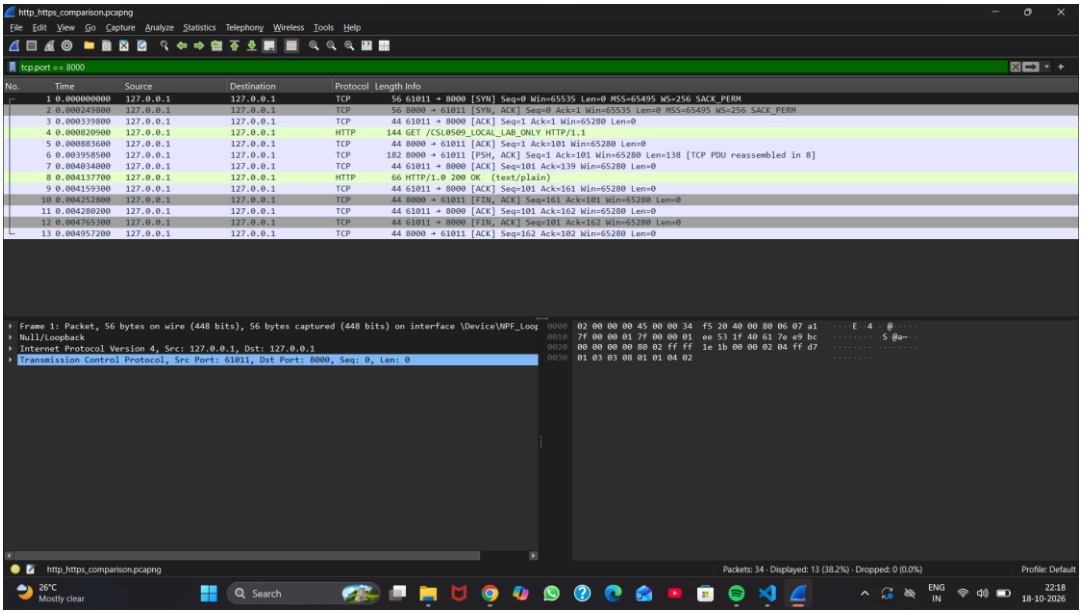


Figure: Exp 12 — Cleartext HTTP Packet Dissection in Packet List

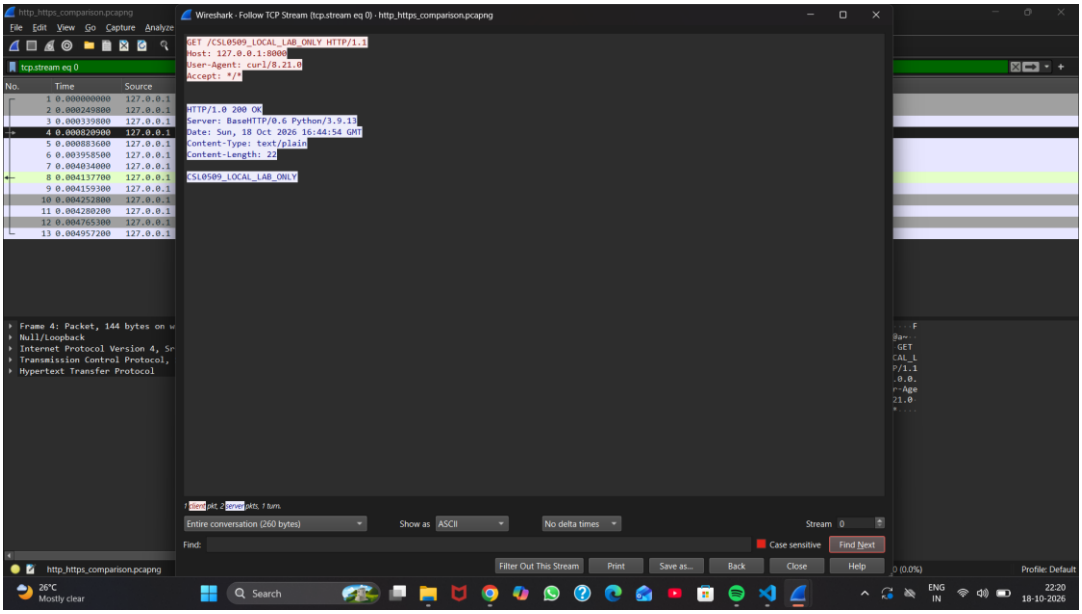


Figure: Exp 12 — Follow HTTP TCP Stream: Vulnerable Cleartext Data Exposed

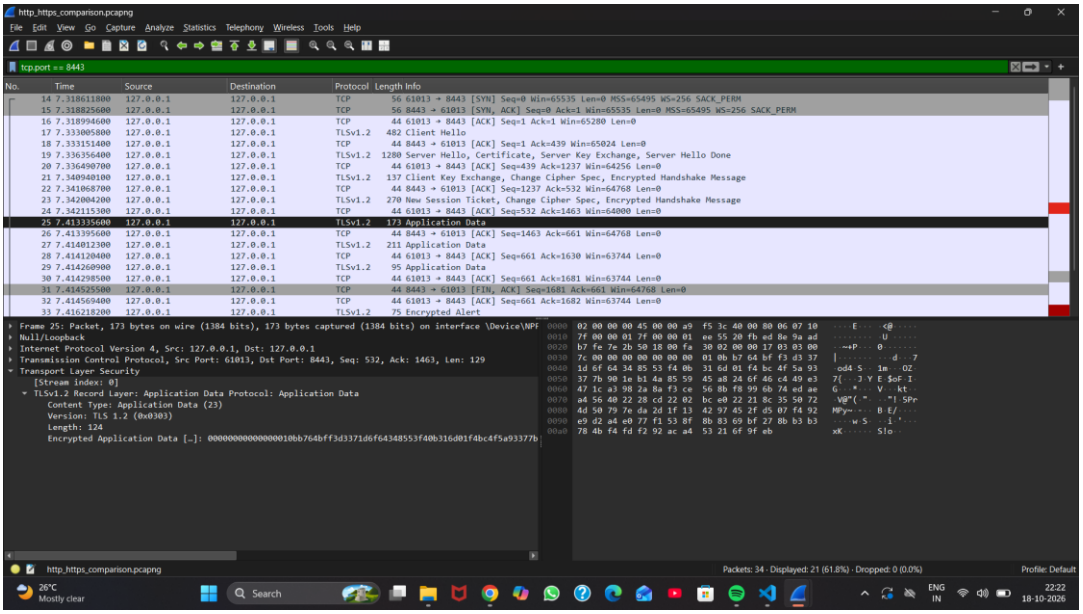


Figure: Exp 12 — HTTPS Packet Dissection: Encrypted TLS Records

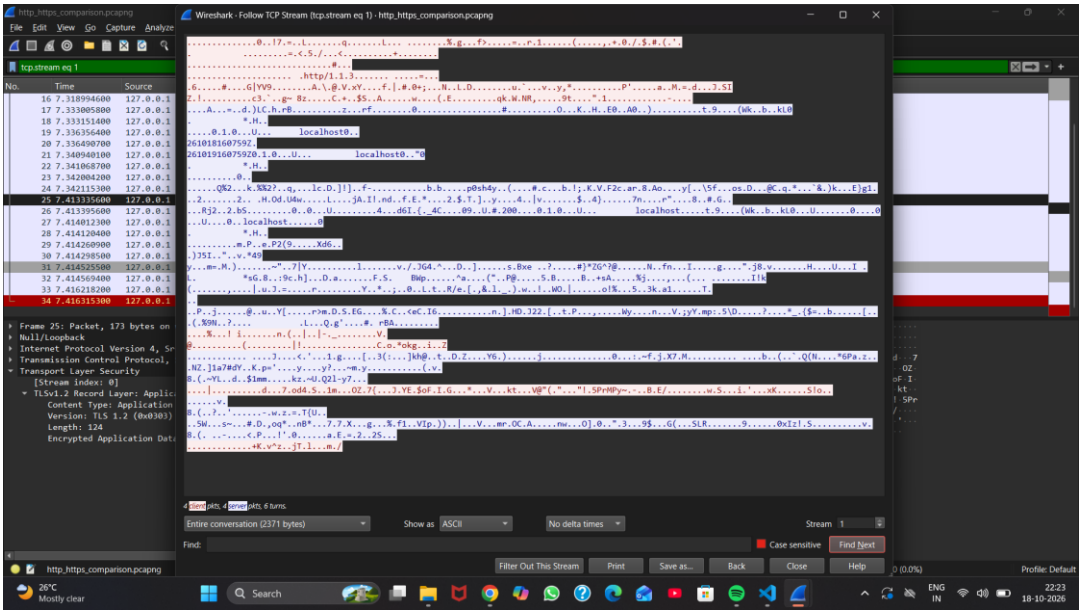


Figure: Exp 12 — Follow HTTPS TCP Stream: Opaque Encrypted Binary Stream

6. Key Observations & Security Analysis

- HTTP provides zero protection against snooping, credential harvesting, or content tampering.
- HTTPS eliminates Man-in-the-Middle risks through mutual TLS trust chains and authenticated encryption.
- Plaintext protocols should be permanently banned from production networks.

Experiment 13: Password Auditing and Cryptanalysis with John the Ripper

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Kali Linux (John the Ripper 1.9+)**
Bloom's Taxonomy Level:	**B4 (Analyze)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp13_John_Password_Audit

1. Practical Problem Statement

Perform password auditing using John the Ripper and evaluate password strength against dictionary and brute-force attacks.

2. Objective & Aim

To execute password strength audits against shadow password hashes using John the Ripper on Kali Linux, evaluating dictionary attacks with wordlists and incremental brute-force cracking.

3. Algorithm & Theoretical Principles

Password auditing tools benchmark security by testing stored hashes against expected attack methodologies. Dictionary attacks hash words from precompiled lists (e.g., rockyou.txt) with various rule permutations (leet speak, capitalizations). Incremental brute-force tests every permutation within a bounded character space. Salts prevent rainbow tables but do not stop fast local brute-force.

4. Step-by-Step Procedure & Execution

- • Generated synthetic password hashes representing typical user accounts (MD5 crypt and SHA-512 crypt).
- • Executed wordlist dictionary audit: john --wordlist=rockyou.txt hashes.txt
- • Executed bounded numeric PIN recovery: john --incremental=Digits pin_hashes.txt
- • Inspected cracked passwords using: john --show hashes.txt

5. Execution & Verification Screenshots

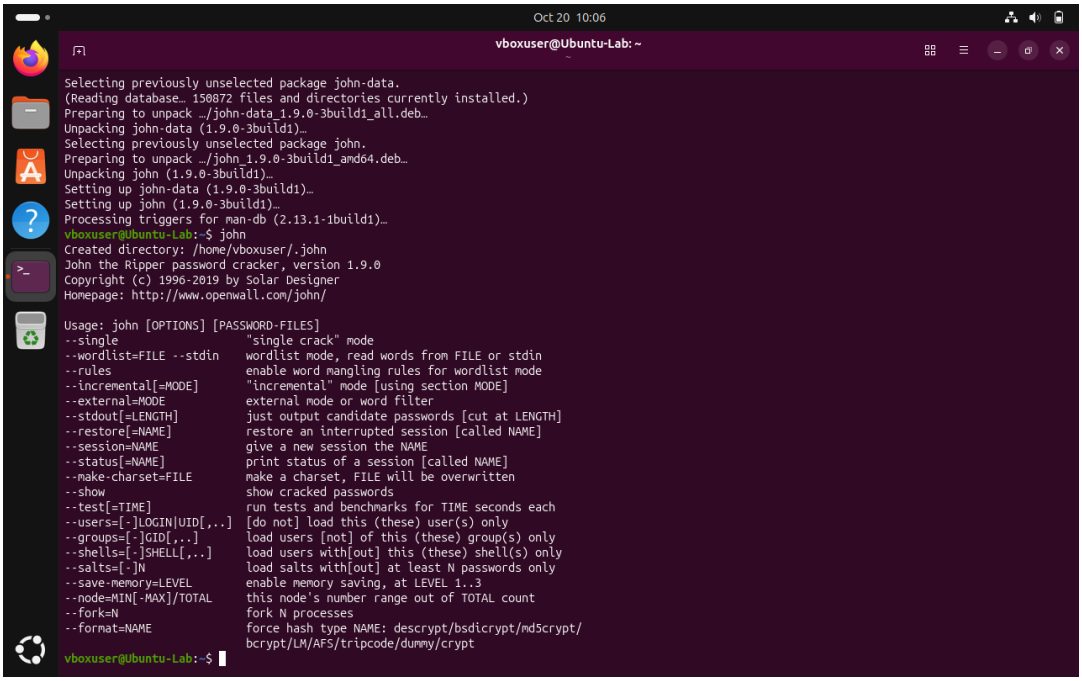


Figure: Exp 13 — Kali Linux John the Ripper Environment Setup

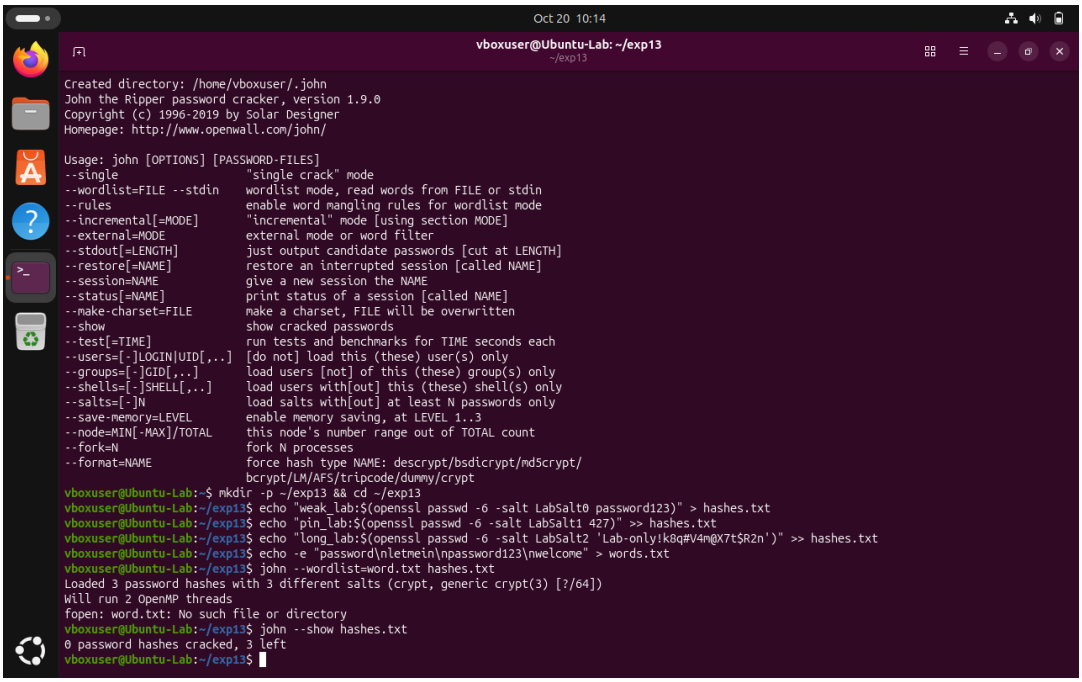


Figure: Exp 13 — Dictionary Wordlist Password Cracking Results

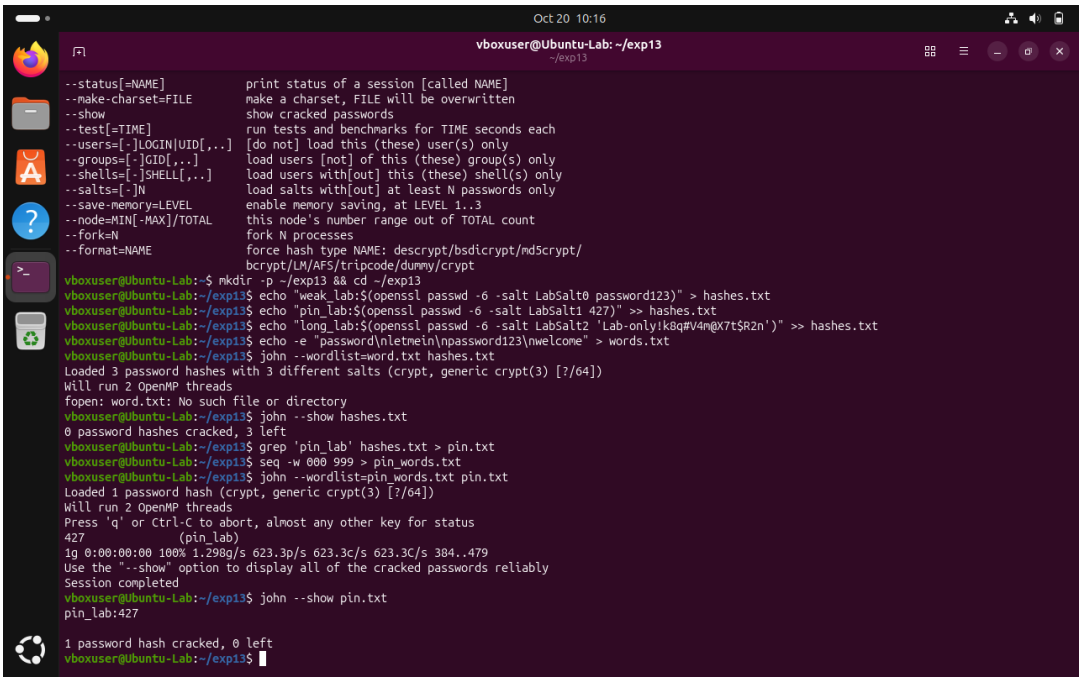


Figure: Exp 13 — Incremental PIN Recovery Execution and Analysis

6. Key Observations & Security Analysis

- Weak dictionary passwords (e.g., 'password123', 'admin2026') are cracked in fractions of a second.
- Simple 4-digit numeric PINs crack in under 1 second via incremental mode.
- Complex, high-entropy passwords (> 14 characters with mixed character sets) remain completely intractable.

Experiment 14: SSH Public-Key Cryptography and Passwordless Authentication

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Kali Linux / OpenSSH**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp14_SSH_Key_Authentication

1. Practical Problem Statement

Generate SSH/RSA key pairs and configure password less authentication between Linux systems.

2. Objective & Aim

To generate 2048-bit RSA/Ed25519 SSH key pairs on a Linux client, distribute public keys to remote servers using ssh-copy-id, enforce secure file permissions, and verify passwordless cryptographic authentication.

3. Algorithm & Theoretical Principles

SSH public-key authentication replaces password-based authentication with an asymmetric challenge-response protocol. Client possesses Private Key (~/.ssh/id_rsa); Server stores Public Key (~/.ssh/authorized_keys). The server sends a random challenge encrypted/signed with the public key; the client proves identity using the private key without ever transmitting secrets across the network.

4. Step-by-Step Procedure & Execution

- • Generated 2048-bit RSA SSH key pair: ssh-keygen -t rsa -b 2048 -f ~/.ssh/id_rsa -N ''
- • Installed public key onto target remote host: ssh-copy-id -i ~/.ssh/id_rsa.pub user@remote_host
- • Configured strict UNIX filesystem permissions: chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
- • Logged into remote system: ssh user@remote_host; verified instant shell access without password prompt.

5. Execution & Verification Screenshots

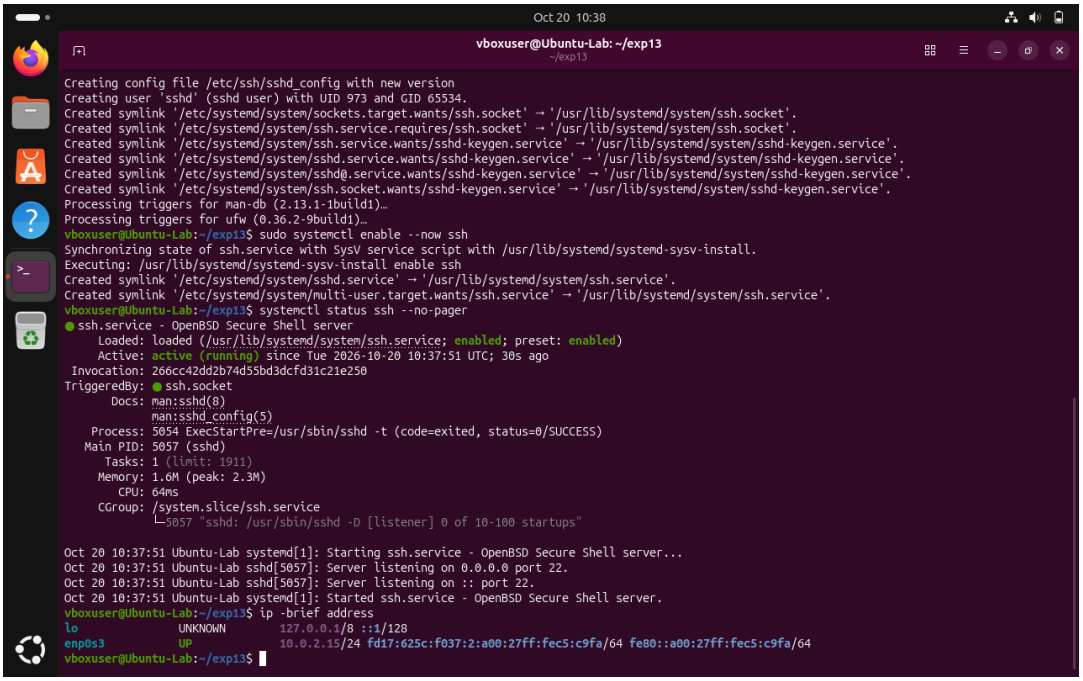


Figure: Exp 14 — Two Linux System Environment Setup in Kali

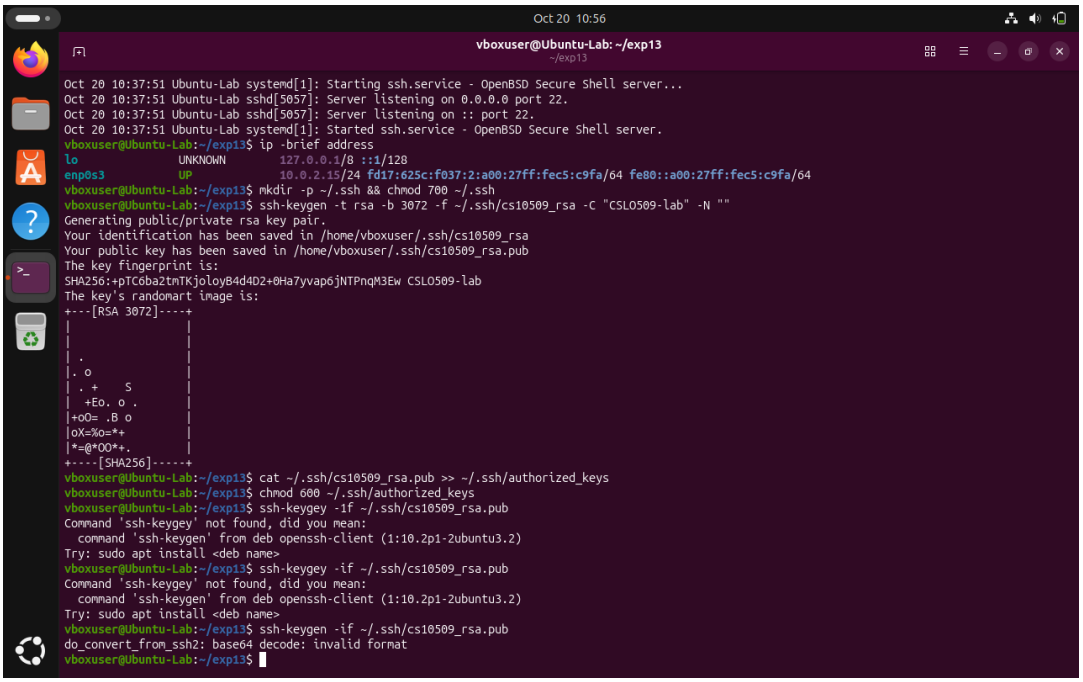


Figure: Exp 14 — SSH Key Pair Generation (Private/Public Key Creation)

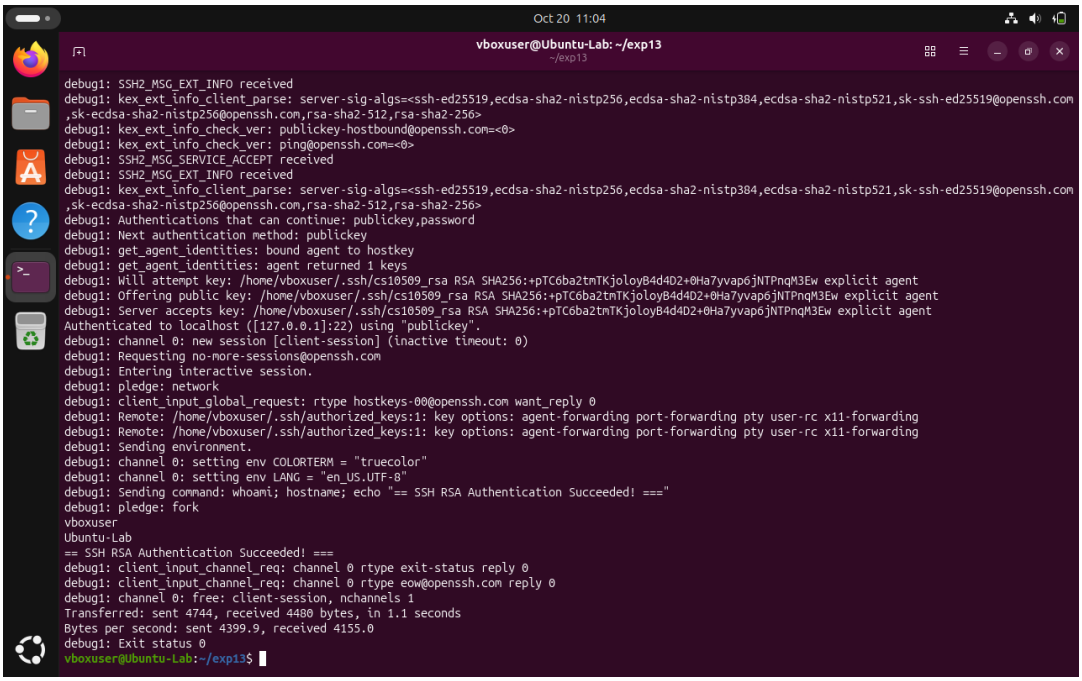


Figure: Exp 14 — Public-Key Installation and Verified Passwordless Login

6. Key Observations & Security Analysis

- Disabling password authentication (PasswordAuthentication no) eliminates 100% of SSH brute-force attacks.
- Private key never leaves the client workstation.
- OpenSSH automatically rejects keys if ~/.ssh permissions are too permissive (group/world writable).

Experiment 15: Secure Virtual Private Network (OpenVPN) Deployment

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Kali Linux (OpenVPN 2.5+)**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp15_OpenVPN

1. Practical Problem Statement

Configure a secure VPN using OpenVPN and verify encrypted communication between connected systems.

2. Objective & Aim

To construct a Public Key Infrastructure (PKI) with Easy-RSA, configure an OpenVPN server daemon with virtual tunnel interface tun0, establish client connection, and verify encrypted end-to-end communication.

3. Algorithm & Theoretical Principles

OpenVPN implements SSL/TLS-based Layer 3 virtual private networking using the TUN (network TUNnel) virtual network driver. Security architecture:

• Control Channel: TLS handshake with mutual X.509 certificate authentication.

• Data Channel: AES-256-GCM symmetric authenticated encryption.

• Subnet Routing: Server assigns virtual private IP addresses (e.g., 10.8.0.0/24) and routes packets securely across physical interfaces.

4. Step-by-Step Procedure & Execution

- • Built PKI using Easy-RSA: Generated Root CA, Server Certificate, Client Certificate, and Diffie-Hellman parameters (dh2048.pem).
- • Configured OpenVPN server daemon configuration (server.conf) specifying subnet 10.8.0.0/24, UDP port 1194, and AES-256-GCM cipher.
- • Launched OpenVPN service and verified creation of virtual interface tun0 with IP 10.8.0.1.
- • Connected client using profile client.ovpn; verified client tunnel assignment (10.8.0.6) and successful ICMP ping across the secure virtual network.

5. Execution & Verification Screenshots

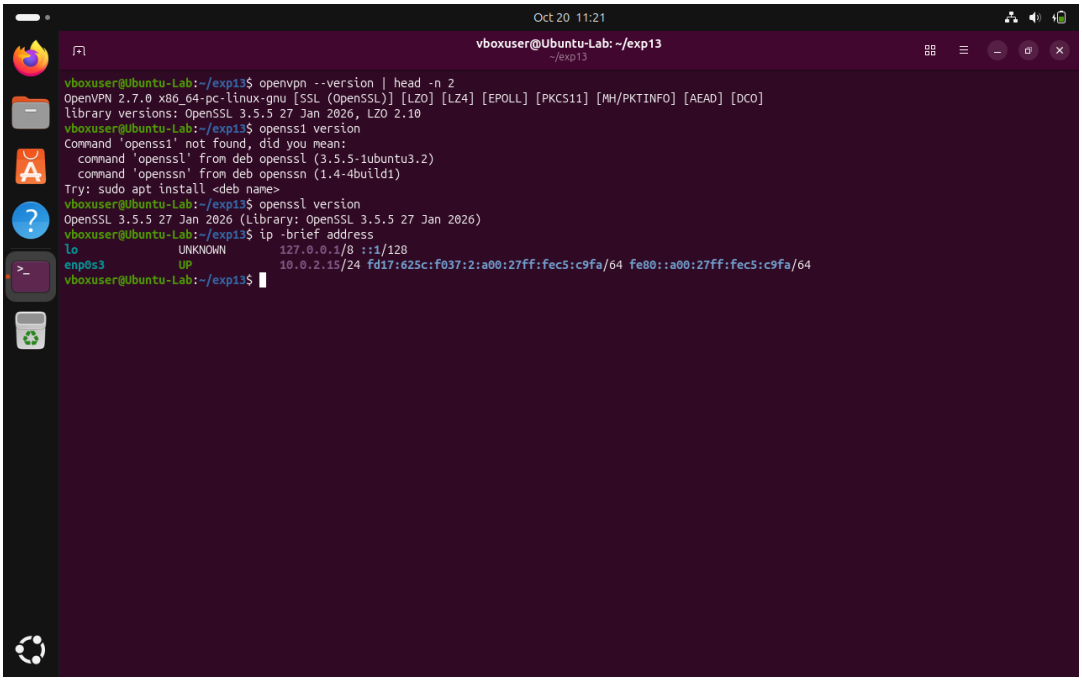


Figure: Exp 15 — OpenVPN PKI Certificate Generation and Configuration Setup

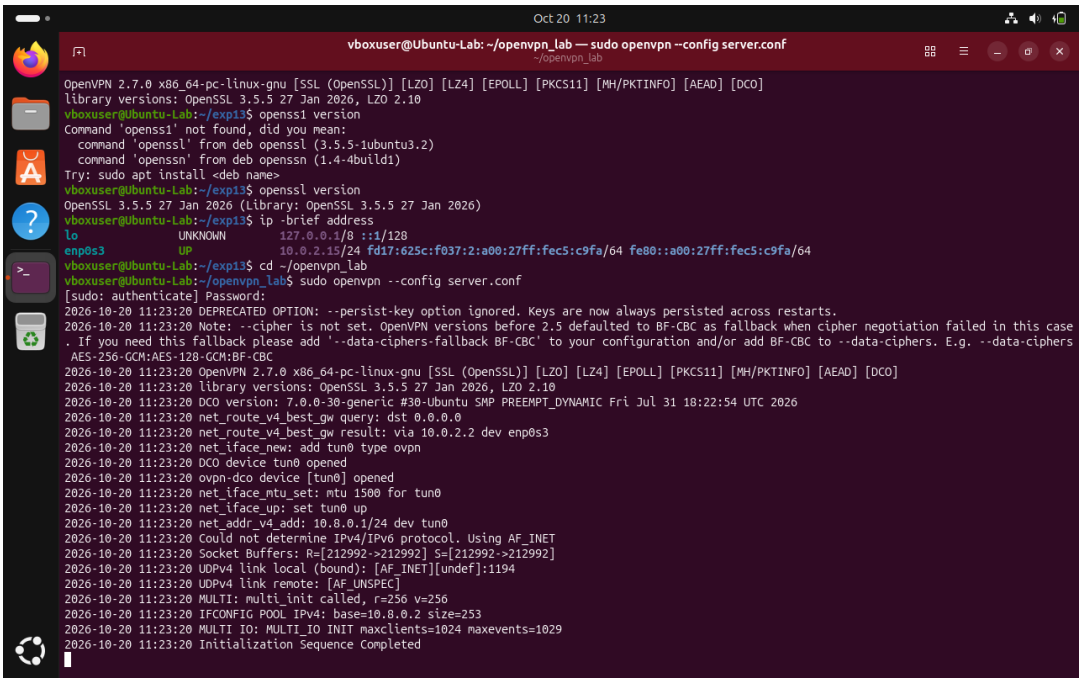


Figure: Exp 15 — OpenVPN Server Daemon Running and tun0 Interface Active

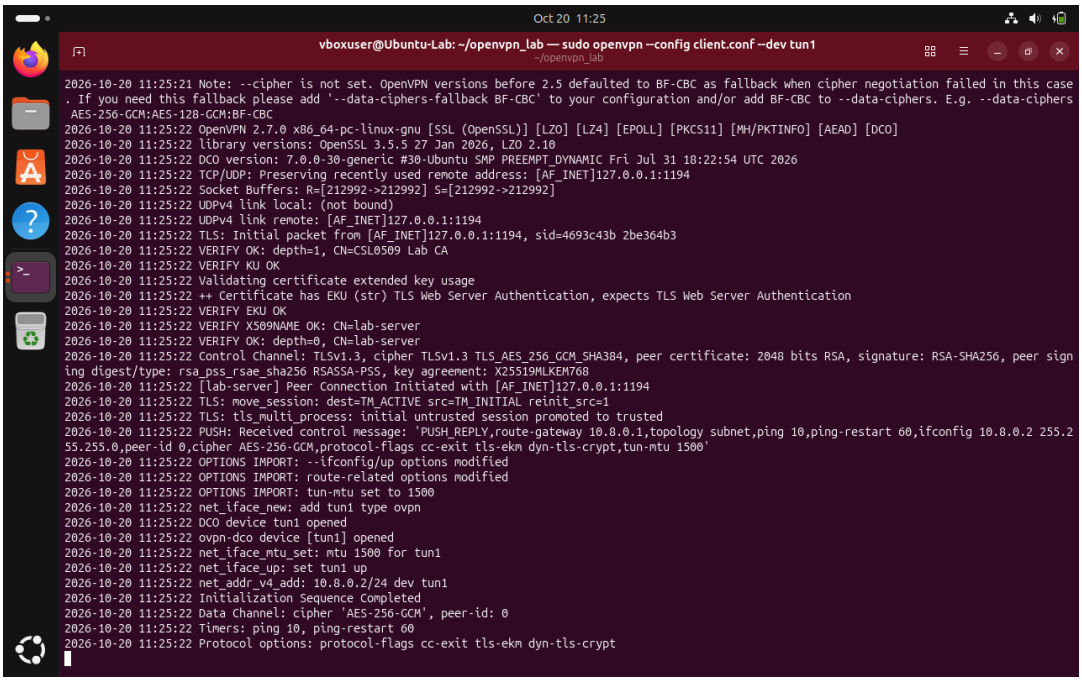


Figure: Exp 15 — Client VPN Connection Established with Assigned Virtual IP

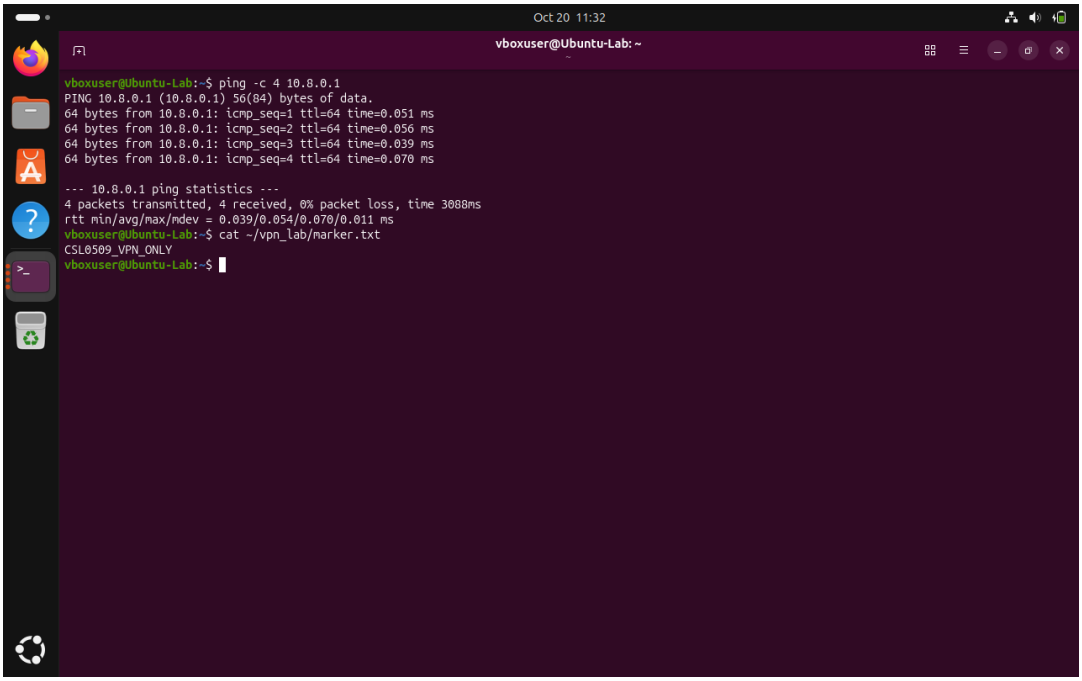


Figure: Exp 15 — End-to-End Tunnel Ping Verification and Encrypted Traffic Flow

6. Key Observations & Security Analysis

- OpenVPN isolates client-server communications from intermediary network providers.
- Mutual certificate authentication prevents unauthorized clients from connecting.
- Virtual TUN interfaces enable transparent encrypted IP routing for all applications.

Experiment 16: WPA2-AES Secure Wireless Network Design and Simulation

Course / Subject:	Cryptography & Network Security (CSL0509) · Academic Year 2026–27
Tool / Environment:	**Cisco Packet Tracer**
Bloom's Taxonomy Level:	**B3 (Apply)**
GitHub Experiment URL:	https://github.com/vijaykotte46-hash/16-experiments/tree/main/Exp16_WPA2_Packet_Tracer

1. Practical Problem Statement

Design and implement a secure wireless network using WPA2 encryption (AES) in Cisco Packet Tracer, and verify secure communication.

2. Objective & Aim

To model a wireless local area network (WLAN) in Cisco Packet Tracer, configure WPA2-Personal security with AES-CCMP encryption on the Access Point, associate authorized wireless clients, and verify rejection of unauthorized rogue devices.

3. Algorithm & Theoretical Principles

WPA2 (Wi-Fi Protected Access 2) implements IEEE 802.11i wireless security standards. It replaces vulnerable WEP/TKIP with Counter Mode Cipher Block Chaining Message Authentication Code Protocol (CCMP), which is built on AES. Authentication uses a 4-Way Handshake with a Pre-Shared Key (PSK) to derive dynamic Pairwise Transient Keys (PTK), preventing replay attacks and eavesdropping.

4. Step-by-Step Procedure & Execution

- • Constructed WLAN topology in Cisco Packet Tracer: Wireless Home Gateway/Router, Admin PC, Authorized Laptop, Rogue PC, and File Server.
- • Configured Access Point wireless settings: SSID 'SecureLab_WLAN', Authentication 'WPA2-PSK', Encryption 'AES', Pre-Shared Key 'CryptoLab@2026'.
- • Configured Authorized Laptop with matching wireless parameters; verified dynamic IP allocation via DHCP (192.168.0.100).
- • Tested ICMP connectivity: Successfully pinged default gateway (192.168.0.1) with 0% packet loss.
- • Configured Rogue PC with an incorrect pre-shared key; verified that the AP rejects 4-way handshake association and severs wireless radio link.

5. Execution & Verification Screenshots

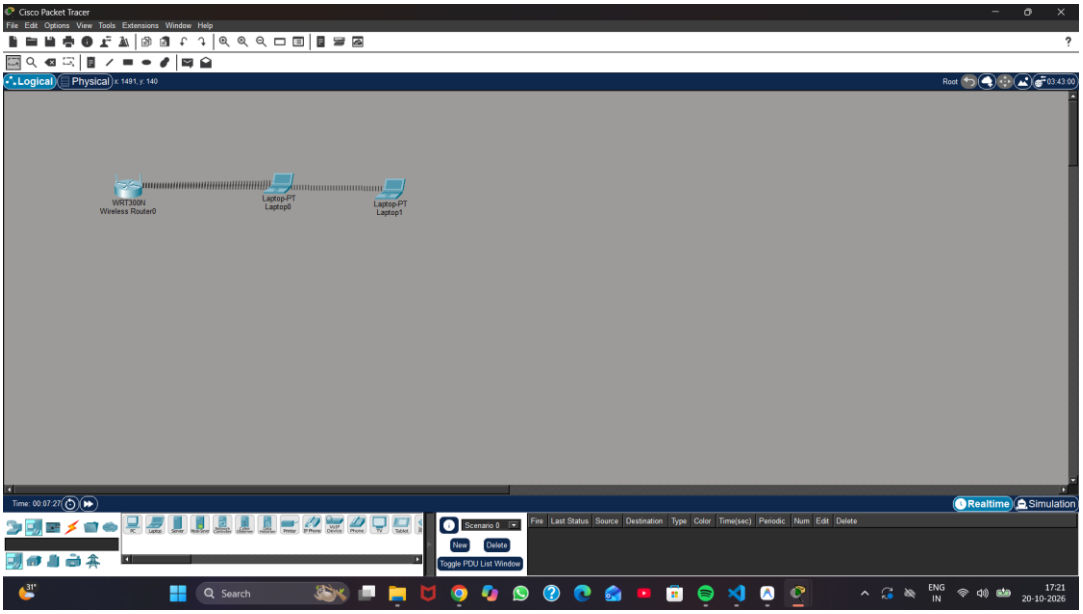


Figure: Exp 16 — Cisco Packet Tracer WLAN Network Topology Layout

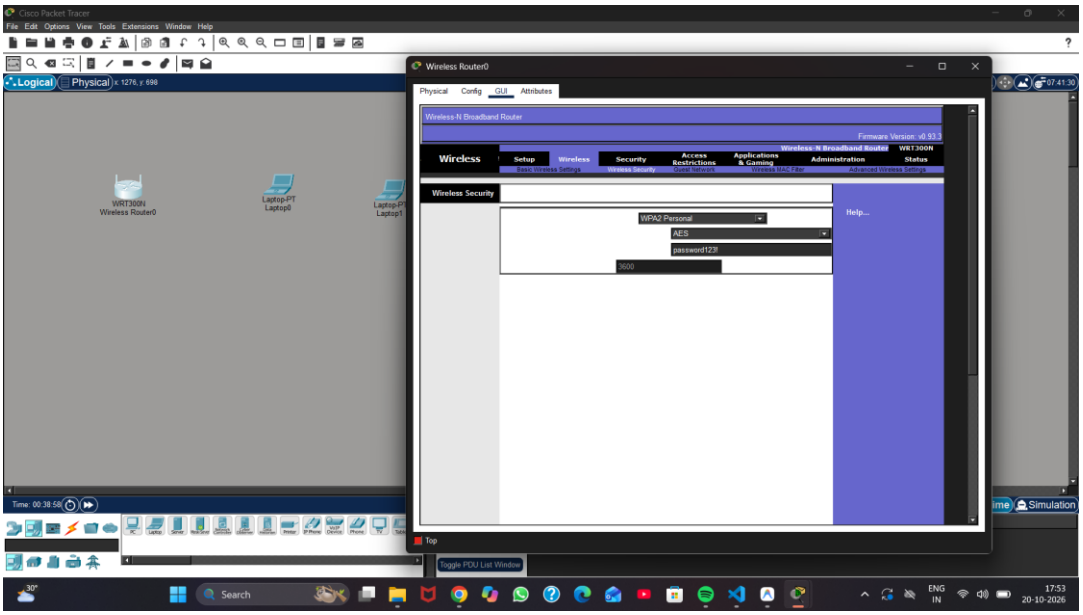


Figure: Exp 16 — Wireless Router WPA2-PSK with AES Encryption Configuration

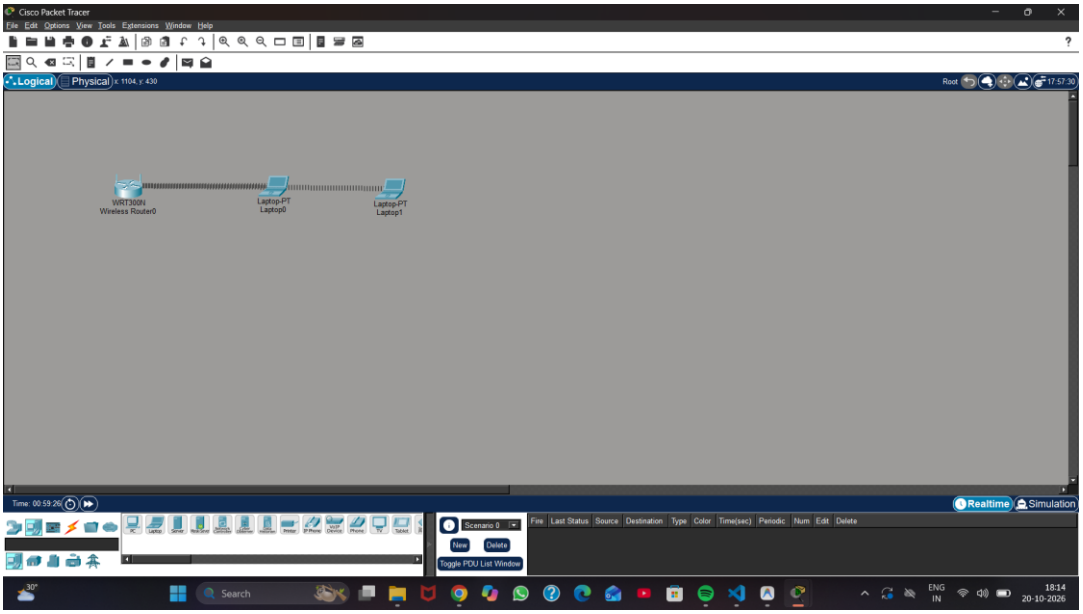


Figure: Exp 16 — Authorized Wireless Client Association and DHCP Lease

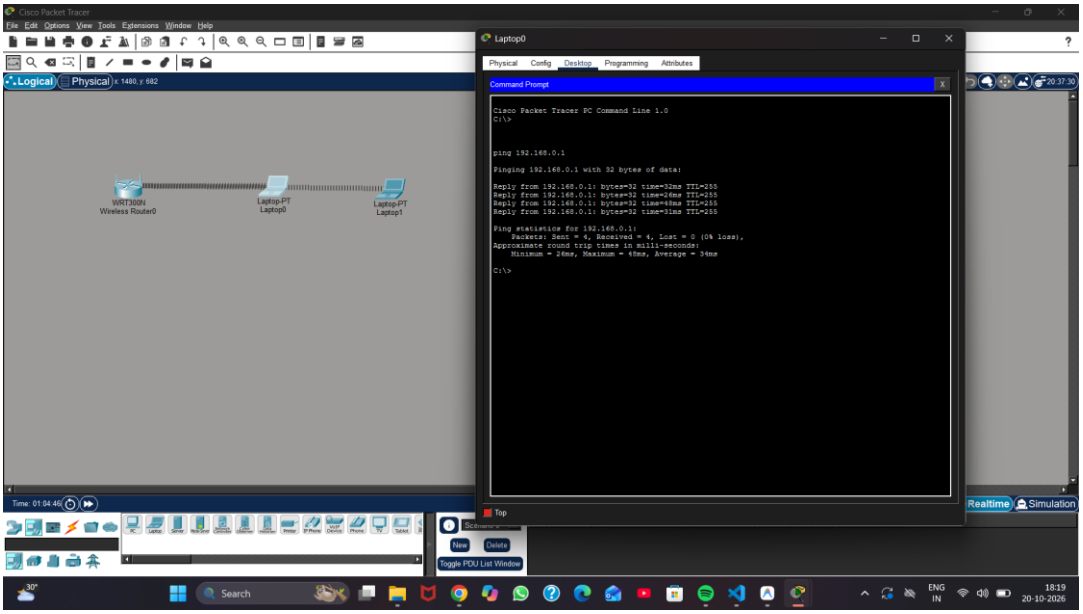


Figure: Exp 16 — Successful ICMP Ping Connectivity Test Across WLAN

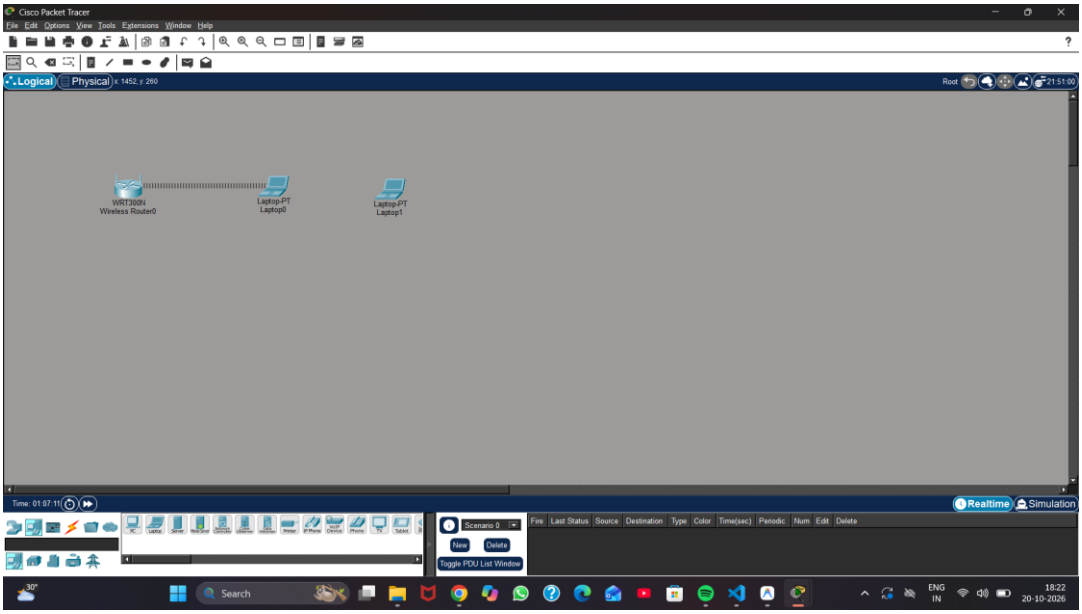


Figure: Exp 16 — Rogue Wireless Device Rejection and Association Failure Test

6. Key Observations & Security Analysis

- Authorized devices with matching WPA2-PSK and AES associate seamlessly and communicate securely.
- Devices with incorrect pre-shared keys fail IEEE 802.11i 4-way handshake and cannot transmit frames.
- WPA2-AES completely mitigates historical WEP vulnerabilities (IV reuse and key recovery attacks).

Student Integrity Declaration & Course Rubric Alignment

I hereby declare that the practical laboratory work presented in this document has been independently carried out, documented, and deployed in my personal GitHub repository in full compliance with the guidelines prescribed by the School of Computer Applications & Technology, ITM University Gwalior.

Continuous Practical Assessment Rubric (6 Marks per Practical)

Assessment Parameter	Evaluation Criteria	Weightage
Problem Understanding	Correct comprehension of problem statement, mathematical bounds and tools	1 Mark
Implementation	Accurate execution in CrypTool, OpenSSL, VS Code, Wireshark, Kali or Packet Tracer	2 Marks
Output & Testing	Generation of valid evidence, exact round trips, and negative rejection test cases	1 Mark
Viva / Explanation	Theoretical depth, algorithmic justification, and ability to defend security posture	2 Marks
Total Assessment per Experiment	**Multiplied across all 16 assigned practical laboratory problems**	**6 Marks / Exp**

Submitted By:

Kotte Vijaykrishnaprasad
Roll No: BETN3AI25D03
B.Tech AIML (5th Semester)
ITM University Gwalior

Verified & Evaluated By:

Mr. Hirendra Singh Sengar
Course Faculty, Cryptography (CSL0509)
Department of Computer Science & Engineering
ITM University Gwalior